

**T. C. SAKARYA UYGULAMALI BİLİMLER
ÜNİVERSİTESİ**

TEKNOLOJİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

SİSTEM PROGRAMLAMA PROJE ÖDEVİ

RISC V TABANLI ASSEMBLER VE LİNKER SİSTEMİ

TASARIMI VE FPGA ÜZERİNDE GERÇEKLENMESİ

Ders Sorumlusu : Prof. Dr.HALİT ÖZTEKİN

Hazırlayanlar :

CHARAF EDDINE BESSANANE	B210109591
CANER PEKKAN	22010903101
ERINC PIRILDAR	B200109023
BEYZA NİMET ONER	22010903036

MAYIS 2026

ÖZET	i
ŞEKİLLER LİSTESİ	ii
TABLolar LİSTESİ	iii
EKRAN GÖRÜNTÜLERİ LİSTESİ	iv
1. GİRİŞ	1
2. LİTERATÜR ARAŞTIRMASI	3
3. SİSTEM MİMARİSİ	5
3.1 Genel Sistem Mimarisi	5
3.2 Two-Pass Assembler Yapısı	6
3.3 Relocation Süreci	7
3.4 FPGA Bellek Yapısı	8
4. KULLANILAN VERİ YAPILARI	9
4.1 Opcode Table	9
4.2 Symbol Table	10
4.3 ESTAB	11
5. ASSEMBLER ALGORİTMASI	12
6. LİNKER TASARIMI	18
7. FPGA ÜZERİNDE ÇALIŞTIRMA	24
8. TEST SENARYOLARI VE SONUÇLAR ...	30
9. ALGORİTMA KARMAŞIKLIĞI ANALİZİ ...	39
10. MÜHENDİSLİK ÇÖZÜMÜNÜN ETKİLERİ ...	43
11. TAKIM ÇALIŞMASI	47
12. ETİK VE HUKUKSAL UYUM	51
13. YAŞAM BOYU ÖĞRENME	55
14. SONUÇ	59
15. KAYNAKÇA	62

ŞEKİLLER LİSTESİ

Şekil 3.1. Genel Sistem Mimarisi Diyagramı

Şekil 3.2. Two-Pass Assembler Çalışma Diyagramı

Şekil 3.3. Relocation ve Symbol Resolution Diyagramı

Şekil 3.4. FPGA Bellek Yerleşim Şeması

Şekil 11.1. Takım Çalışması Süreç Dağılımı

TABLolar LİSTESİ

Tablo 8.1. Instruction Encoding Test Sonuçları

Tablo 8.2. Object Dosya ve Linkleme Test Sonuçları

Tablo 8.3. FPGA Simülasyon Test Sonuçları

Tablo 11.1. Proje Toplantı Süreçleri

EKRAN GÖRÜNTÜLERİ LİSTESİ

Ekran Görüntüsü 4.1. Opcode Table Yapısı

Ekran Görüntüsü 4.2. Symbol Table Çıktısı

Ekran Görüntüsü 4.3. ESTAB Yapısı

Ekran Görüntüsü 4.4. Object Dosya Yapısı

Ekran Görüntüsü 5.1. Parser Sistemi Çalışması

Ekran Görüntüsü 5.3. Object Dosya Üretim Çıktısı

Ekran Görüntüsü 5.5. Assembler Arayüzü

Ekran Görüntüsü 6.1. MAIN.obj Dosyası

Ekran Görüntüsü 6.2. LOGIC.obj Dosyası

Ekran Görüntüsü 6.3. IODRV.obj Dosyası

Ekran Görüntüsü 6.4. ESTAB Yapısı

Ekran Görüntüsü 6.5. Linkleme Sonrası HEX Çıktısı

Ekran Görüntüsü 7.1. pico_bram.mem Dosyası

Ekran Görüntüsü 7.2. Verilog Simülasyon Çalıştırılması

Ekran Görüntüsü 7.4. Assembler Arayüzü

Ekran Görüntüsü 7.5. Linker Çalıştırma Arayüzü

Ekran Görüntüsü 8.2. Object Dosya Test Çıktısı

Ekran Görüntüsü 8.5. EXTREF Etiket Uzunluğu Hatası

Ekran Görüntüsü 8.6. EXTDEF Etiket Uzunluğu Hatası

Ekran Görüntüsü 8.7. Negatif Başlangıç Adresi Testi

Ekran Görüntüsü 8.8. Başlangıç Adresi Taşma Testi

Ekran Görüntüsü 8.10. Aynı Etiketin Birden Fazla Tanımlanması

ÖZET

Bu projede PicoRV32 işlemcisi için RV32I komut kümesini destekleyen assembler ve linker sistemi geliştirilmiştir. Geliştirilen sistem assembly seviyesinde yazılmış kaynak kodları object code formatına dönüştürmekte ve farklı object dosyalarını birleştirerek FPGA üzerinde çalıştırılabilir executable çıktı üretmektedir.

Assembler sistemi içerisinde parser, opcode table, symbol table, machine code generation ve object file generation modülleri geliştirilmiştir. Linker sistemi içerisinde ise external symbol resolution, ESTAB yapısı, relocation mekanizmaları ve address patching algoritmaları gerçekleştirilmiştir. Ayrıca oluşturulan executable çıktılar FPGA BRAM belleğine yüklenerek PicoRV32 işlemcisi üzerinde çalıştırılmıştır.

Projede modüler yazılım geliştirme yaklaşımı kullanılmış, instruction encoding mekanizmaları RV32I mimarisine uygun şekilde tasarlanmış ve relocation işlemleri object dosyaları arasında başarıyla gerçekleştirilmiştir. Geliştirilen sistem farklı test senaryoları ile doğrulanmış ve FPGA simülasyon ortamında başarılı sonuçlar elde edilmiştir.

Bu proje kapsamında sistem programlama, assembler tasarımı, linker algoritmaları, FPGA tabanlı gömülü sistemler, bellek yönetimi ve düşük seviyeli yazılım geliştirme alanlarında mühendislik deneyimi kazanılmıştır.

Anahtar Kelimeler: Assembler, Linker, PicoRV32, RV32I, FPGA, Relocation, ESTAB, Object Code

1. GİRİŞ

Assembler ve linker yazılımları bilgisayar sistemlerinin temel sistem yazılımı bileşenleri arasında yer almaktadır. Assembler sistemleri assembly seviyesinde yazılmış kaynak kodları işlemci tarafından yürütülebilecek machine code formatına dönüştürürken, linker sistemleri ise birbirinden bağımsız şekilde oluşturulmuş object dosyalarını birleştirerek çalıştırılabilir executable dosya üretmektedir.

Modern bilgisayar sistemlerinde assembler ve linker yapıları özellikle gömülü sistemler, işletim sistemleri, FPGA tabanlı mimariler ve işlemci geliştirme süreçlerinde kritik öneme sahiptir. Instruction encoding, relocation işlemleri, symbol resolution mekanizmaları ve bellek yönetimi düşük seviyeli sistem yazılımlarının temel problemleri arasında yer almaktadır.

Son yıllarda açık kaynak RISC-V mimarisinin yaygınlaşması ile birlikte bu mimariye yönelik sistem yazılımı geliştirme çalışmaları akademik ve endüstriyel alanlarda önem kazanmıştır. Özellikle FPGA tabanlı sistemlerde düşük kaynak tüketimine sahip RISC-V işlemci çekirdekleri esnek ve düşük maliyetli çözümler sunmaktadır.

Bu proje kapsamında PicoRV32 işlemcisi temel alınarak RV32I komut kümesini destekleyen modüler yapıda bir assembler ve linker sistemi geliştirilmiştir. Geliştirilen assembler sistemi assembly kaynak kodlarını object code formatına dönüştürmekte, linker sistemi ise farklı object dosyalarını birleştirerek FPGA üzerinde çalıştırılabilir executable çıktı üretmektedir.

Assembler sistemi içerisinde parser, opcode table, symbol table, instruction encoder, machine code generation ve object file generation modülleri geliştirilmiştir. Linker sistemi içerisinde ise ESTAB yapısı, external symbol resolution mekanizması, relocation işlemleri, address patching algoritmaları ve final executable generation süreçleri gerçekleştirilmiştir.

Geliştirilen sistem FPGA tabanlı PicoRV32 işlemcisi üzerinde test edilmiş ve oluşturulan executable çıktılar BRAM belleğine yüklenerek başarıyla çalıştırılmıştır. Ayrıca sistem üzerinde farklı test senaryoları uygulanmış, branch relocation işlemleri doğrulanmış ve object dosyaları arasında external symbol resolution mekanizmalarının doğru çalıştığı gözlemlenmiştir.

Bu proje kapsamında ařağıdaki mühendislik kazanımları elde edilmiştir:

- Sistem seviyesinde yazılım geliştirme becerisi
- RV32I instruction encoding mekanizmalarının öğrenilmesi
- Assembly dili analiz süreçleri
- Linker algoritmalarının uygulanması
- Relocation işlemlerinin gerçekleştirilmesi
- FPGA tabanlı gömülü sistem geliştirme süreçleri
- Bellek yönetimi ve adresleme işlemleri
- Teknik test ve doğrulama süreçleri
- Açık kaynak sistem geliştirme yaklaşımı

Proje geliştirme süreci boyunca modüler yazılım mimarisi yaklaşımı benimsenmiş, veri yapıları optimize edilmiş ve sistemin genişletilebilir olması hedeflenmiştir. Ayrıca geliştirilen sistem farklı object dosyaları üzerinde test edilmiş ve relocation mekanizmalarının doğruluğı FPGA simülasyon ortamında doğrulanmıştır.

(PÇ6, PÇ7, PÇ13, PÇ17)

2. LİTERATÜR ARAŞTIRMASI

Assembler ve linker sistemleri bilgisayar bilimleri ve sistem programlama alanlarının en temel yazılım bileşenleri arasında yer almaktadır. İlk assembler sistemleri doğrudan makine kodu üretmek amacıyla geliştirilmiş olup zamanla modern işlemci mimarilerinin gelişmesiyle birlikte daha karmaşık ve modüler yapılara dönüşmüştür.

Günümüzde GNU Assembler (GAS), NASM, MASM ve LLVM tabanlı assembler sistemleri yaygın şekilde kullanılmaktadır. Bu sistemler farklı işlemci mimarilerini desteklemekte ve assembly seviyesindeki kaynak kodları işlemci tarafından çalıştırılabilir machine code formatına dönüştürmektedir.

GNU Assembler özellikle Linux tabanlı sistemlerde yaygın şekilde kullanılmakta olup ELF object formatı, relocation mekanizmaları ve external symbol management gibi gelişmiş özellikleri desteklemektedir. NASM ise x86 mimarisi için geliştirilen modüler assembler sistemlerinden biridir. LLVM altyapısı ise modern compiler ve assembler sistemlerinde optimize edilmiş code generation süreçleri sağlamaktadır.

RISC-V mimarisi açık kaynak instruction set architecture yaklaşımı ile geliştirilen modern işlemci mimarilerinden biridir. Açık kaynak olması nedeniyle akademik çalışmalarda, araştırma projelerinde ve FPGA tabanlı sistemlerde yaygın şekilde kullanılmaktadır. RISC-V mimarisinin modüler yapısı farklı işlemci tasarımlarının geliştirilmesini kolaylaştırmaktadır.

Bu proje kapsamında kullanılan PicoRV32 işlemcisi Clifford Wolf tarafından geliştirilen hafif yapıli bir RISC-V işlemci çekirdeğidir. PicoRV32 düşük kaynak tüketimi, sade instruction pipeline yapısı ve FPGA sistemleri ile uyumlu mimarisi nedeniyle gömülü sistem projelerinde tercih edilmektedir.

Assembler sistemlerinde yaygın olarak kullanılan yaklaşım two-pass assembler algoritmasıdır. Birinci geçişte symbol table oluşturulurken ikinci geçişte machine code generation işlemleri gerçekleştirilmektedir. Bu yaklaşım özellikle forward reference problemlerinin çözümünde önemli avantaj sağlamaktadır.

Forward reference problemi assembly programı içerisinde henüz adresi bilinmeyen label ifadelerinin kullanılması durumunda ortaya çıkmaktadır. Two-pass yaklaşımı sayesinde ilk geçişte bütün label adresleri belirlenmekte, ikinci geçişte ise instruction encoding işlemleri doğru adres bilgileri kullanılarak gerçekleştirilmektedir.

Linker sistemleri object dosyalarının birleştirilmesi, external sembollerin çözülmesi, relocation işlemleri ve executable dosya üretiminden sorumludur. Modern linker sistemleri özellikle relocation mekanizmaları sayesinde bağımsız şekilde derlenmiş modüllerin birlikte çalıştırılabilmesini sağlamaktadır.

Relocation işlemleri sırasında object dosyaları içerisinde bulunan geçici adresler final executable oluşturulurken yeniden düzenlenmektedir. Böylece farklı object dosyalarının farklı bellek bölgelerine yerleştirilmesi mümkün hale gelmektedir.

Bu projede geliştirilen linker sistemi aşağıdaki temel mekanizmaları kullanmaktadır:

- External symbol management
- ESTAB (External Symbol Table) yapısı
- DEF ve REF kayıtları
- Relocation (RLC) mekanizmaları
- Address patching algoritmaları
- HEX/memory file generation süreçleri

Object dosyaları içerisinde kullanılan DEF kayıtları ilgili object dosyasında tanımlanan global sembolleri ifade etmektedir. REF kayıtları ise farklı object dosyaları içerisinde çözümlenecek external sembolleri göstermektedir. Linker sistemi ESTAB yapısı aracılığıyla bu sembollerin adres çözümü işlemlerini gerçekleştirmektedir.

Relocation mekanizmaları özellikle branch ve jump instruction'ları için kritik öneme sahiptir. Çünkü object dosyaları oluşturulurken final bellek adresleri bilinmemektedir. Linker sistemi relocation işlemleri sırasında instruction offset değerlerini yeniden hesaplayarak executable dosya oluşturmaktadır.

FPGA tabanlı gömülü sistemlerde assembler ve linker sistemlerinin kullanımı düşük seviyeli sistem yazılımlarının geliştirilmesi açısından büyük önem taşımaktadır. Özellikle BRAM memory initialization süreçleri executable dosyaların FPGA işlemcileri üzerinde çalıştırılabilmesini sağlamaktadır.

Bu projede geliştirilen sistem literatürde kullanılan temel assembler ve linker yaklaşımlarını uygulamakta ve PicoRV32 işlemcisi üzerinde FPGA tabanlı executable generation süreçlerini gerçekleştirmektedir. Özellikle ESTAB tabanlı symbol resolution yapısı, relocation algoritmaları ve object dosya yönetimi projenin temel mühendislik bileşenleri arasında yer almaktadır.

(PÇ6, PÇ13, PÇ17)

3. SİSTEM MİMARİSİ

Bu bölümde geliştirilen assembler ve linker sisteminin genel mimari yapısı, modülleri, veri akışı ve FPGA entegrasyon süreçleri detaylı şekilde açıklanmaktadır.

Geliştirilen sistem modüler yazılım geliştirme yaklaşımı kullanılarak tasarlanmıştır. Modüler mimari sayesinde assembler ve linker sistemleri birbirinden bağımsız şekilde geliştirilebilmekte, test edilebilmekte ve genişletilebilmektedir. Bu yaklaşım aynı zamanda hata ayıklama süreçlerini kolaylaştırmakta ve sistemin sürdürülebilirliğini artırmaktadır.

Sistem temel olarak dört ana katmandan oluşmaktadır:

1. Kaynak Kod İşleme Katmanı
2. Object Kod Üretim Katmanı
3. Linkleme ve Relocation Katmanı
4. FPGA Çalıştırma Katmanı

Kaynak kod işleme katmanı assembly dosyalarının okunması, ayrıştırılması ve yorumlanmasından sorumludur. Object kod üretim katmanı machine code generation işlemlerini gerçekleştirmektedir. Linkleme katmanı object dosyalarının birleştirilmesini ve relocation işlemlerini yürütmektedir. FPGA katmanı ise final executable çıktısının PicoRV32 işlemcisi üzerinde çalıştırılmasını sağlamaktadır.

Assembler sistemi assembly kaynak dosyalarını satır satır analiz ederek instruction parsing işlemlerini gerçekleştirmektedir. Parser modülü instruction türlerini belirlemekte, register operandlarını analiz etmekte ve immediate değerleri ayrıştırmaktadır. Daha sonra encoder modülü RV32I instruction encoding kurallarına uygun şekilde machine code üretmektedir.

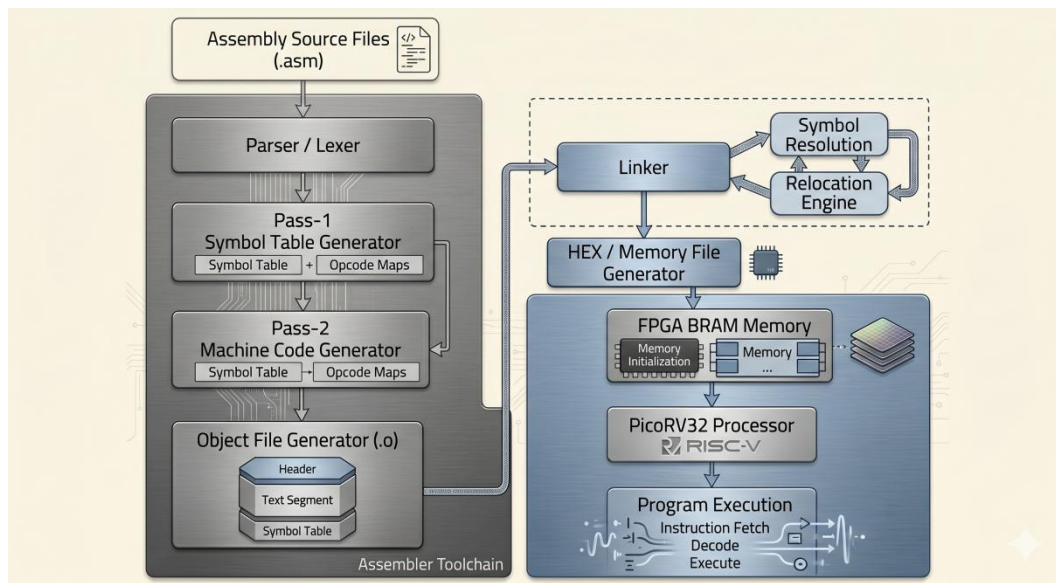
Assembler sistemi içerisinde two-pass çalışma mantığı kullanılmaktadır. Birinci geçişte symbol table oluşturulmakta ve label adresleri hesaplanmaktadır. İkinci geçişte ise instruction encoding işlemleri gerçekleştirilmekte ve object dosyası oluşturulmaktadır.

Linker sistemi assembler tarafından oluşturulan object dosyalarını analiz ederek DEF, REF ve RLC kayıtlarını işlemektedir. Daha sonra ESTAB yapısı oluşturulmakta ve external sembollerin gerçek adresleri hesaplanmaktadır.

Relocation işlemleri sırasında branch ve jump instruction'larının offset değerleri yeniden hesaplanmaktadır. Böylece object dosyaları farklı bellek bölgelerine yerleştirilse bile instruction adresleri doğru şekilde güncellenebilmektedir.

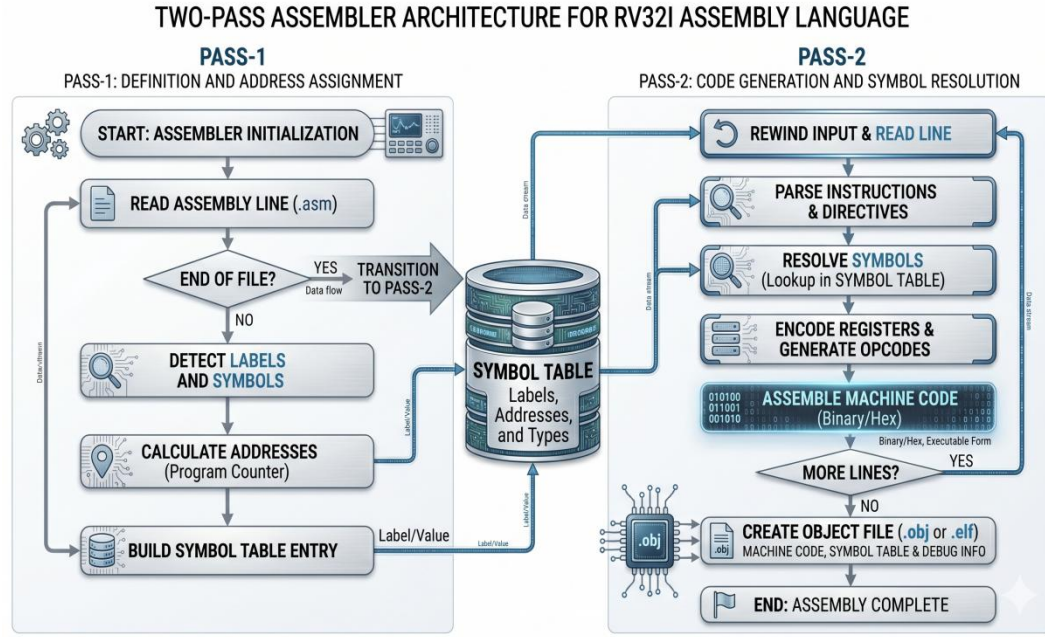
Final executable üretim aşamasında linker sistemi HEX/memory formatında çıktı oluşturmaktadır. Bu çıktı FPGA BRAM belleğine yüklenmekte ve PicoRV32 işlemcisi tarafından çalıştırılmaktadır.

Projede FPGA simülasyon süreçleri Verilog tabanlı test ortamı kullanılarak gerçekleştirilmiştir. Simülasyon sırasında instruction execution, memory erişimleri, branch işlemleri ve relocation sonuçları doğrulanmıştır.



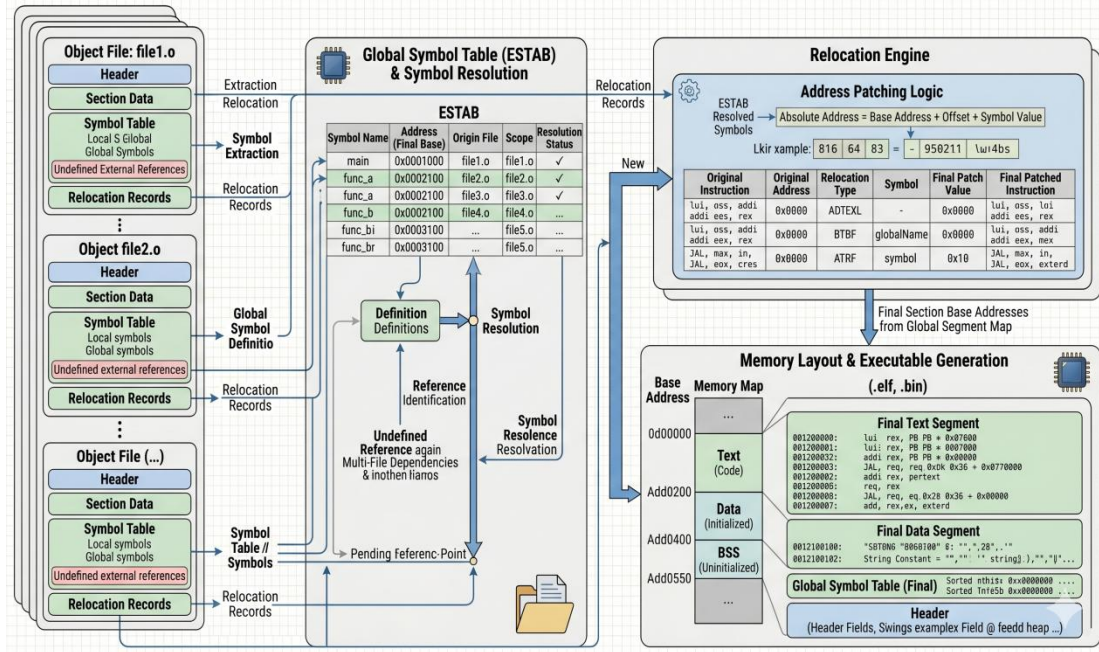
Şekil 3.1. Genel Sistem Mimarisi Diyagramı

Şekil 3.1'de geliştirilen assembler ve linker sisteminin genel mimarisi gösterilmektedir. Sistem assembly kaynak dosyalarının parser modülü tarafından analiz edilmesi ile başlamaktadır. Daha sonra assembler sistemi machine code ve object dosyaları üretmektedir. Linker sistemi farklı object dosyalarını birleştirerek relocation işlemlerini gerçekleştirmekte ve FPGA üzerinde çalıştırılabilir executable çıktı oluşturmaktadır. Son aşamada oluşturulan HEX/memory dosyası FPGA BRAM belleğine yüklenerek PicoRV32 işlemcisi üzerinde çalıştırılmaktadır.



Şekil 3.2. Two-Pass Assembler Çalışma Diyagramı

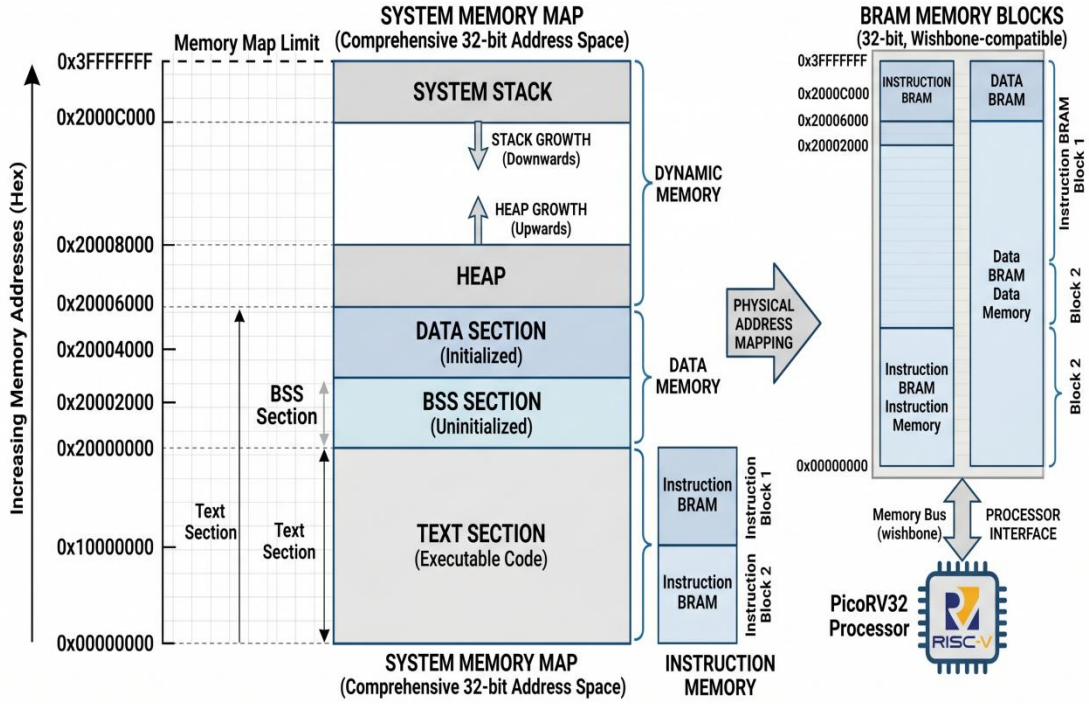
Şekil 3.2’de assembler sistemi içerisinde kullanılan two-pass çalışma yapısı gösterilmektedir. Birinci geçiş aşamasında assembly dosyası analiz edilmekte, label adresleri belirlenmekte ve symbol table oluşturulmaktadır. İkinci geçişte ise opcode dönüşümleri gerçekleştirilmekte, instruction encoding işlemleri uygulanmakta ve machine code üretimi yapılmaktadır. Two-pass yaklaşımı forward reference problemlerinin çözülmesini sağlamaktadır.



Şekil 3.3. Relocation ve Symbol Resolution Diyagramı

Bu şekilde linker sistemi içerisinde gerçekleştirilen relocation ve symbol resolution süreçleri gösterilmektedir. Linker farklı object dosyaları içerisindeki DEF ve REF kayıtlarını analiz etmekte, ESTAB yapısını oluşturmakta ve external sembollerin gerçek adreslerini hesaplamaktadır. Daha sonra relocation işlemleri sırasında instruction offset değerleri güncellenmektedir.

PicoRV32 RISC-V FPGA System Memory Layout Diagram



Şekil 3.4. FPGA Bellek Yerleşim Şeması

Şekil 3.4'te FPGA belleği içerisindeki text section, data section ve instruction memory bölgeleri gösterilmektedir. Linker sistemi final executable çıktısını FPGA BRAM yapısına uygun şekilde yerleştirmektedir. Böylece PicoRV32 işlemcisi instruction fetch işlemlerini doğru adreslerden gerçekleştirebilmektedir.

4. KULLANILAN VERİ YAPILARI

Assembler ve linker sistemlerinin doğru, hızlı ve güvenilir şekilde çalışabilmesi için çeşitli veri yapıları kullanılmıştır. Bu veri yapıları assembly komutlarının analiz edilmesi, sembollerin yönetilmesi, object dosyalarının oluşturulması ve linker işlemlerinin gerçekleştirilmesi sırasında kritik görev üstlenmektedir.

Geliştirilen sistem içerisinde temel olarak aşağıdaki veri yapıları kullanılmıştır:

- Opcode Table
- Symbol Table
- External Symbol Table (ESTAB)

Bu veri yapıları sistemin modüler ve genişletilebilir şekilde tasarlanmasını sağlamaktadır. Ayrıca instruction lookup, symbol resolution ve relocation işlemlerinin düşük zaman karmaşıklığı ile gerçekleştirilmesine yardımcı olmaktadır.

4.1 Opcode Table

Opcode table assembler sisteminin temel veri yapılarından biridir. Bu yapı RV32I komut kümesinde bulunan assembly komutlarının opcode, funct3 ve funct7 bilgilerini tutmak amacıyla kullanılmaktadır.

Assembler sistemi machine code generation aşamasında assembly komutlarını opcode table üzerinden analiz ederek ilgili binary karşılıklarını üretmektedir. Böylece instruction encoding işlemleri merkezi bir yapı üzerinden gerçekleştirilmektedir.

Opcode table yapısının kullanılmasının temel avantajları aşağıdaki gibidir:

- Komut dönüşüm işlemlerinin hızlandırılması
- Kod tekrarının azaltılması
- Yeni komut desteğinin kolaylaştırılması
- Merkezi instruction management yapısının oluşturulması
- Instruction encoding süreçlerinin sadeleştirilmesi

Bu projede opcode table yapısı dictionary/hash map benzeri veri yapıları kullanılarak gerçekleştirilmiştir. Böylece instruction lookup işlemleri düşük zaman karmaşıklığı ile gerçekleştirilebilmektedir.

Opcode table içerisinde aşağıdaki bilgiler tutulmaktadır:

- Instruction mnemonic
- Opcode bilgisi
- Instruction formatı
- Funct3 alanı
- Funct7 alanı

Bu yapı sayesinde assembler sistemi farklı instruction türlerini doğru şekilde encode edebilmektedir.

```
6 class Encoder:
7     # =====
8     # OPCODES = {
9     #
10    # "R-TYPE": 0x33,
11    # "I-TYPE": 0x13,
12    #
13    # "LW": 0x03,
14    # "SW": 0x23,
15    #
16    # "BRANCH": 0x63,
17    #
18    # "JAL": 0x6F
19    # }
20    # =====
21    # FUNCT3
22    # =====
23    # FUNCT3 = {
24    #
25    # "ADD": 0x0,
26    # "SUB": 0x0,
27    # "AND": 0x7,
28    # "OR": 0x6,
29    # "XOR": 0x4,
30    #
31    # "ADDI": 0x0,
32    # "ANDI": 0x7,
33    # "ORI": 0x6,
34    #
35    # "LW": 0x2,
36    # "SW": 0x2,
37    #
38    # "BEQ": 0x0,
39    # "BNE": 0x1
40    # }
41    # =====
42    # =====
```

Ekran Görüntüsü 4.1. Opcode Table Yapısı

Bu görüntüde assembler sistemi içerisinde kullanılan opcode table yapısı gösterilmektedir. Tablo içerisinde RV32I instruction'larının opcode, funct3 ve funct7 bilgileri tutulmaktadır. Assembler sistemi machine code generation aşamasında bu tabloyu kullanarak instruction encoding işlemlerini gerçekleştirmektedir.

4.2 Symbol Table

Symbol table assembler sisteminin en önemli veri yapılarından biridir. Bu yapı assembly programı içerisinde tanımlanan label isimlerini ve bu label'lara karşılık gelen adres bilgilerini saklamak amacıyla kullanılmaktadır.

Assembler sistemi birinci geçiş (Pass-1) aşamasında assembly dosyasını analiz ederek program counter değerlerini hesaplamakta ve label adreslerini symbol table içerisine kaydetmektedir.

Symbol table yapısı özellikle aşağıdaki işlemler için kullanılmaktadır:

- Branch instruction adres çözümleme işlemleri
- Jump instruction adres hesaplamaları
- Forward reference problemlerinin çözülmesi
- Label kontrol mekanizmaları
- Program counter yönetimi

Forward reference problemi assembly programı içerisinde henüz adresi bilinmeyen label ifadelerinin kullanılması durumunda ortaya çıkmaktadır. Two-pass assembler yaklaşımı sayesinde ilk geçişte bütün label adresleri belirlenmekte ve ikinci geçişte instruction encoding işlemleri doğru adres bilgileri ile gerçekleştirilmektedir.

Bu yapı sayesinde assembly kodları içerisinde kullanılan label ifadeleri doğru adreslere dönüştürülebilmektedir.

output.txt			output.mem		
Fichier	Modifier	Affichage	Fichier	Modifier	Affichage
PROJE: IODRV			PROJE: LOGIC		
ETIKET		ADRES	ETIKET		ADRES
-----			-----		
WRLED		000000	GET42		000000

Ekran Görüntüsü 4.2. Symbol Table Çıktısı

Bu görüntüde assembler sistemi tarafından oluşturulan symbol table yapısı gösterilmektedir. Tablo içerisinde label isimleri ve ilgili adres bilgileri tutulmaktadır. Özellikle branch ve jump instruction'larının adres çözümleme işlemleri sırasında bu yapı kullanılmaktadır.

4.3 External Symbol Table (ESTAB)

Linker sistemi içerisinde farklı object dosyaları arasında bulunan external sembollerin yönetilebilmesi amacıyla ESTAB (External Symbol Table) veri yapısı geliştirilmiştir.

Assembler sistemi tarafından oluşturulan object dosyaları linker aşamasında analiz edilmekte ve global semboller ESTAB yapısı içerisine kaydedilmektedir. Daha sonra symbol resolution ve relocation işlemleri sırasında bu yapı kullanılmaktadır.

Object dosyaları içerisinde kullanılan DEF kayıtları ilgili object dosyasında tanımlanan global sembollerini ifade etmektedir. REF kayıtları ise farklı object dosyaları içerisinde çözümlenecek external sembollerini göstermektedir.

Linker sistemi bütün object dosyalarını analiz ederek ESTAB yapısını oluşturmaktadır. Daha sonra relocation işlemleri sırasında gerekli instruction adresleri bu yapı kullanılarak güncellenmektedir.

ESTAB yapısının temel görevleri aşağıdaki gibidir:

- External sembollerin adres bilgilerinin tutulması
- Object dosyaları arasında sembol eşleştirmesi yapılması
- Relocation işlemleri sırasında adres çözümleme yapılması
- JAL ve branch instruction'larının doğru hedef adreslere yönlendirilmesi
- Çoklu object dosyası desteğinin sağlanması

Bu yaklaşım sayesinde bağımsız şekilde derlenmiş object dosyaları güvenli şekilde birleştirilebilmektedir.

Fichier Modifier Affichage H1 ▾ ≡ ▾ B I

EXTERNAL SYMBOL TABLE

MODULE	SYMBOL	ADDRESS
IODRV	<u>IODRV</u>	000000
IODRV	WRLED	000000
LOGIC	<u>LOGIC</u>	000008
LOGIC	GET42	000008
MAIN	<u>MAIN</u>	000010
MAIN	RE1	000014
MAIN	HALT1	000018

Ekran Görüntüsü 4.3. ESTAB Yapısı

Bu görüntüde linker sistemi tarafından oluşturulan ESTAB yapısı gösterilmektedir. Tablo içerisinde external sembol isimleri, adres bilgileri ve ilgili object dosya referansları tutulmaktadır. Linker sistemi relocation işlemleri sırasında bu yapı üzerinden adres çözümleme gerçekleştirmektedir.

Fichier	Modifier	Affichage
H^IODRV ^000000^000008		
D^WRLED ^000000		
R^HALT1		
T^000000^08^08A020230000006F		
M^000004^R_RISCV_JAL^HALT1		
E^000000		

Ekran Görüntüsü 4.4. Object Dosya Yapısı

Bu görüntüde assembler sistemi tarafından oluşturulan object dosyası yapısı gösterilmektedir. Object dosyası içerisinde DEF, REF, DAT ve RLC kayıtları bulunmaktadır. DAT kayıtları machine code verilerini içerirken, RLC kayıtları relocation işlemleri sırasında güncellenecek instruction adreslerini göstermektedir.

5. ASSEMBLER ALGORİTMASI

Bu bölümde geliştirilen assembler sisteminin çalışma mantığı, instruction parsing süreçleri, machine code generation mekanizmaları ve object dosya üretim algoritmaları detaylı şekilde açıklanmaktadır.

Assembler sistemi assembly seviyesinde yazılmış kaynak kodları analiz ederek RV32I instruction set architecture yapısına uygun machine code üretmektedir. Geliştirilen sistem modüler mimari yaklaşımı kullanılarak tasarlanmış olup parser, encoder, symbol management ve object generation modüllerinden oluşmaktadır.

Assembler sistemi temel olarak aşağıdaki aşamalardan oluşmaktadır:

1. Kaynak dosyanın okunması
2. Lexer ve parser işlemleri
3. Label analizleri
4. Symbol table oluşturulması
5. Instruction parsing işlemleri
6. Machine code generation
7. Object dosyası oluşturulması

Assembler sistemi içerisinde two-pass çalışma mantığı kullanılmaktadır. Birinci geçişte symbol table oluşturulurken ikinci geçişte machine code generation işlemleri gerçekleştirilmektedir. Bu yaklaşım forward reference problemlerinin çözülmesini sağlamaktadır.

5.1 Lexer ve Parser Sistemi

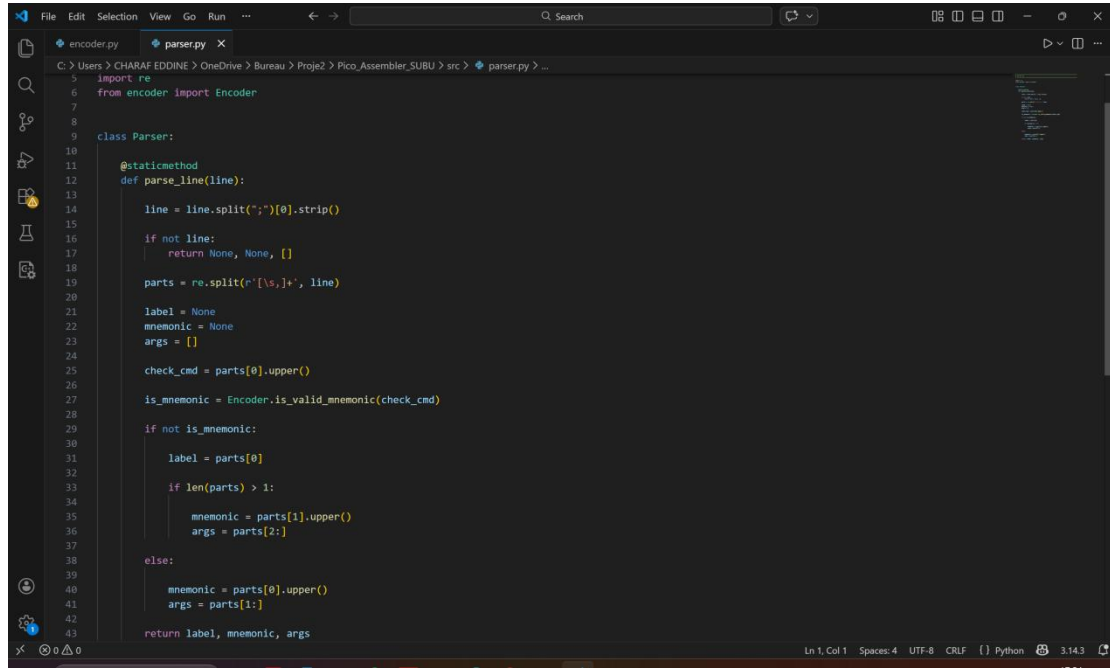
Assembler sistemi ilk olarak assembly kaynak dosyasını satır satır okumaktadır. Okunan satırlar parser modülü tarafından analiz edilmekte ve instruction bileşenlerine ayrıştırılmaktadır.

Parser sistemi aşağıdaki bileşenleri analiz etmektedir:

- Instruction mnemonic
- Register operandları
- Immediate değerler
- Label ifadeleri
- Direktifler
- Bellek adresleme yapıları

Instruction parsing işlemleri sırasında assembly komutları tokenize edilmekte ve ilgili instruction formatına göre yorumlanmaktadır.

Parser sistemi aynı zamanda syntax kontrol işlemlerini de gerçekleştirmektedir. Hatalı instruction kullanımları, eksik operandlar veya geçersiz register ifadeleri hata yönetim sistemi tarafından raporlanmaktadır.



```
5 import re
6 from encoder import Encoder
7
8
9 class Parser:
10
11     @staticmethod
12     def parse_line(line):
13
14         line = line.split(";")[0].strip()
15
16         if not line:
17             return None, None, []
18
19         parts = re.split(r'[\s,]+', line)
20
21         label = None
22         mnemonic = None
23         args = []
24
25         check_cmd = parts[0].upper()
26
27         is_mnemonic = Encoder.is_valid_mnemonic(check_cmd)
28
29         if not is_mnemonic:
30
31             label = parts[0]
32
33             if len(parts) > 1:
34
35                 mnemonic = parts[1].upper()
36                 args = parts[2:]
37
38             else:
39
40                 mnemonic = parts[0].upper()
41                 args = parts[1:]
42
43         return label, mnemonic, args
```

Ekran Görüntüsü 5.1. Parser Sistemi Çalışması

Bu görüntüde assembler sistemi içerisinde kullanılan parser modülünün çalışma yapısı gösterilmektedir. Parser sistemi assembly satırlarını analiz ederek instruction mnemonic, register operandları ve immediate değerlerini ayrıştırmaktadır.

5.2 Two-Pass Assembler Yapısı

Assembler sistemi two-pass çalışma mantığı kullanılarak geliştirilmiştir. Bu yaklaşım özellikle forward reference problemlerinin çözülmesi açısından kritik öneme sahiptir.

Pass-1 Aşaması

Birinci geçiş aşamasında assembly dosyası analiz edilmekte ve label adresleri belirlenmektedir. Bu aşamada symbol table yapısı oluşturulmaktadır.

Pass-1 aşamasında gerçekleştirilen işlemler:

- Assembly satırlarının okunması
- Label tanımlarının tespit edilmesi
- Program counter hesaplamaları
- Segment analizleri
- Symbol table oluşturulması
- Direktif analizleri

Bu aşamada machine code generation işlemi gerçekleştirilmemektedir.

Pass-2 Aşaması

İkinci geçiş aşamasında instruction encoding işlemleri gerçekleştirilmektedir. Symbol table içerisinde bulunan adres bilgileri kullanılarak machine code üretimi yapılmaktadır.

Pass-2 aşamasında gerçekleştirilen işlemler:

- Instruction parsing
- Opcode lookup işlemleri
- Register encoding işlemleri
- Immediate encoding işlemleri
- Branch offset hesaplamaları
- Machine code generation
- Object dosyası oluşturulması

5.3 Instruction Encoding Mekanizması

Assembler sistemi RV32I instruction set architecture yapısına uygun instruction encoding işlemleri gerçekleştirmektedir. Farklı instruction türleri farklı binary formatlar kullanmaktadır.

Sistem aşağıdaki instruction formatlarını desteklemektedir:

- R-Type
- I-Type
- S-Type
- B-Type
- U-Type
- J-Type

Instruction encoding işlemleri sırasında opcode, funct3, funct7, register alanları ve immediate değerleri ilgili instruction formatına uygun şekilde birleştirilmektedir.

Özellikle branch ve jump instruction'larında offset hesaplamaları dikkatli şekilde gerçekleştirilmektedir. Çünkü relocation işlemleri sırasında bu instruction'ların yeniden güncellenmesi gerekmektedir.

5.4 Direktif Yönetimi

Assembler sistemi assembly programları içerisinde kullanılan direktif yapılarını desteklemektedir. Direktifler assembly programının bellek yerleşimini ve veri organizasyonunu kontrol etmek amacıyla kullanılmaktadır.

Sistem tarafından desteklenen temel direktifler aşağıdaki gibidir:

- .text
- .data
- .word
- .byte
- .org
- .end

.text direktifi instruction section başlangıcını belirtirken .data direktifi veri bölümünü tanımlamaktadır. .word ve .byte direktifleri veri tanımlama işlemleri için kullanılmaktadır.

.org direktifi program counter değerinin değiştirilmesini sağlamakta, .end direktifi ise assembly dosyasının sonunu belirtmektedir

5.5 Object Dosya Üretimi

Assembler sistemi instruction encoding işlemlerinin tamamlanmasının ardından object dosyası oluşturmaktadır. Object dosyaları linker sistemi tarafından kullanılacak ara çıktı formatını temsil etmektedir.

Geliştirilen object dosya yapısı aşağıdaki kayıt türlerini içermektedir:

- OBJ
- DEF
- REF
- DAT
- RLC
- ENDOBJ

OBJ kaydı object modül bilgilerini içermektedir. DEF kayıtları global sembolleri, REF kayıtları external sembolleri temsil etmektedir. DAT kayıtları machine code verilerini içermekte, RLC kayıtları ise

relocation işlemleri sırasında güncellenecek instruction adreslerini göstermektedir.

Object dosya formatı linker sisteminin external symbol resolution ve relocation işlemlerini gerçekleştirebilmesi amacıyla tasarlanmıştır.

Fichier	Modifier	Affichage
H^IODRV	^000000^000008	
D^WRLED	^000000	
R^HALT1		
T^000000^08^08A020230000006F		
M^000004^R_RISCV_JAL^HALT1		
E^000000		

Ekran Görüntüsü 5.3. Object Dosya Üretim Çıktısı

Bu görüntüde assembler sistemi tarafından oluşturulan object dosyası yapısı gösterilmektedir. Object dosyası içerisinde machine code verileri, symbol kayıtları ve relocation bilgileri bulunmaktadır. Linker sistemi bu kayıtları kullanarak executable dosya oluşturmaktadır.

(PÇ6, PÇ7)

5.6 Hata Yönetim Sistemi

Assembler sistemleri düşük seviyeli yazılım geliştirme süreçlerinde çalıştığı için hata yönetim mekanizmaları büyük önem taşımaktadır. Özellikle syntax hataları, geçersiz instruction kullanımları, eksik operand problemleri ve tanımlanmamış semboller assembler sisteminin doğru çalışmasını doğrudan etkilemektedir.

Bu proje kapsamında assembler sistemi içerisine modüler yapıda hata yönetim mekanizması geliştirilmiştir. Geliştirilen sistem assembly dosyalarını analiz ederken oluşabilecek hataları tespit etmekte ve kullanıcıya detaylı hata mesajları sunmaktadır.

Hata yönetim sistemi aşağıdaki hata türlerini kontrol etmektedir:

- Geçersiz instruction kullanımı
- Eksik operand hataları
- Hatalı register kullanımları
- Tanımlanmamış label problemleri
- Geçersiz immediate değerleri
- Syntax hataları
- Desteklenmeyen direktif kullanımları

Parser sistemi instruction analizleri sırasında operand sayılarını ve instruction formatlarını kontrol etmektedir. Hatalı instruction kullanımları tespit edildiğinde hata yönetim sistemi kullanıcıya ilgili satır numarası ile birlikte hata mesajı üretmektedir.

Ayrıca linker sistemi içerisinde de external symbol resolution işlemleri sırasında hata kontrol mekanizmaları kullanılmaktadır. Tanımlanmamış external semboller veya çözümlenemeyen relocation kayıtları linker sistemi tarafından hata olarak raporlanmaktadır.

Bu yaklaşım sistemin güvenilirliğini artırmakta ve hata ayıklama süreçlerini kolaylaştırmaktadır.

5.7 Modüler Yazılım Mimarisi

Geliştirilen assembler sistemi modüler yazılım geliştirme yaklaşımı kullanılarak tasarlanmıştır. Bu yaklaşım sayesinde sistem farklı görevlerden sorumlu bağımsız modüllere ayrılmıştır.

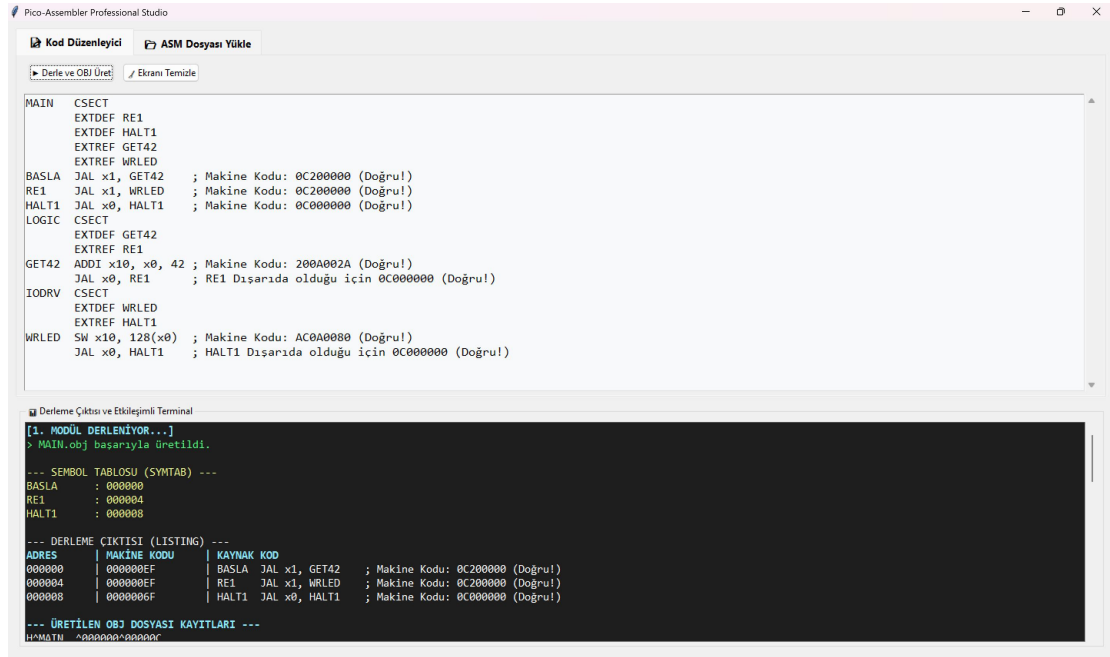
Assembler sistemi içerisinde kullanılan temel modüller aşağıdaki gibidir:

- Parser modülü
- Encoder modülü
- Symbol management modülü
- Object generation modülü
- Hata yönetim modülü
- Dosya yönetim modülü

Parser modülü assembly satırlarını analiz ederek instruction bileşenlerini ayrıştırmaktadır. Encoder modülü RV32I instruction encoding işlemlerini gerçekleştirmektedir. Symbol management modülü label adreslerini yönetmektedir. Object generation modülü ise linker sistemi tarafından kullanılacak object dosyalarını oluşturmaktadır.

Modüler yapı sayesinde sistemin genişletilebilirliği artırılmıştır. Yeni instruction desteği veya yeni object format özellikleri sisteme kolay şekilde eklenebilmektedir.

Ayrıca modüler yaklaşım hata ayıklama süreçlerini kolaylaştırmakta ve kod bakım süreçlerinin daha güvenli şekilde gerçekleştirilmesini sağlamaktadır.



Ekran Görüntüsü 5.5. Assembler Arayüzü

Bu görüntüde geliştirilen assembler sisteminin kullanıcı arayüzü gösterilmektedir. Kullanıcı arayüzü üzerinden assembly dosyaları yüklenebilmekte, assembly işlemleri gerçekleştirilebilmekte ve machine code çıktıları görüntülenebilmektedir.

5.8 Assembler Sisteminin Genel Çalışma Süreci

Assembler sistemi assembly kaynak dosyasının okunması ile başlamaktadır. Parser modülü satır bazlı analiz işlemleri gerçekleştirerek instruction bileşenlerini ayrıştırmaktadır.

Birinci geçiş aşamasında symbol table oluşturulmakta ve label adresleri hesaplanmaktadır. İkinci geçişte ise instruction encoding işlemleri gerçekleştirilmekte ve machine code generation yapılmaktadır.

Assembly işlemleri tamamlandıktan sonra object dosyası oluşturulmaktadır. Oluşturulan object dosyası linker sistemi tarafından kullanılacak ara çıktı formatını temsil etmektedir.

Assembler sistemi tarafından üretilen object dosyaları aşağıdaki bileşenleri içermektedir:

- Object modül bilgileri
- Global semboller
- External semboller
- Machine code verileri
- Relocation kayıtları

Bu yapı linker sisteminin symbol resolution ve relocation işlemlerini güvenli şekilde gerçekleştirebilmesini sağlamaktadır.

Assembler sistemi farklı assembly dosyaları üzerinde test edilmiş ve RV32I instruction encoding işlemlerinin doğru şekilde gerçekleştirildiği doğrulanmıştır.

(PÇ6, PÇ7)

6. LINKER TASARIMI

Bu bölümde geliştirilen linker sisteminin çalışma mantığı, object dosyalarının birleştirilme süreçleri, external symbol resolution mekanizmaları, relocation işlemleri ve executable generation algoritmaları detaylı şekilde açıklanmaktadır.

Linker sistemleri assembler tarafından oluşturulan bağımsız object dosyalarını birleştirerek çalıştırılabilir executable çıktı üretmek amacıyla kullanılmaktadır. Geliştirilen linker sistemi özellikle external symbol management, relocation işlemleri ve FPGA executable generation süreçlerini gerçekleştirmek amacıyla tasarlanmıştır.

Bu projede geliştirilen linker sistemi aşağıdaki temel görevleri yerine getirmektedir:

- Object dosyalarının okunması
- DEF ve REF kayıtlarının analiz edilmesi
- ESTAB yapısının oluşturulması
- External symbol resolution işlemleri
- Relocation işlemleri
- Address patching algoritmaları
- Final executable generation
- HEX/memory file üretimi

Linker sistemi assembler tarafından oluşturulan object dosyalarını analiz ederek external sembollerin gerçek adreslerini hesaplamakta ve relocation işlemlerini gerçekleştirmektedir.

Projede modüler yazılım geliştirme yaklaşımı kullanılmıştır. Assembly programları bağımsız modüller halinde derlenmekte ve her modül için ayrı object dosyası oluşturulmaktadır. Daha sonra linker sistemi bu object dosyalarını birleştirerek final executable çıktısını üretmektedir.

Bu proje kapsamında aşağıdaki object modülleri kullanılmıştır:

- MAIN.obj
- LOGIC.obj
- IODRV.obj

Bu yaklaşım sayesinde sistem farklı modüllere ayrılmış, kod tekrarının önüne geçilmiş ve linker sistemi gerçek çoklu object dosyası mantığı ile geliştirilmiştir.

6.1 Object Dosya Yapısı

Assembler sistemi tarafından oluşturulan object dosyaları linker sistemi tarafından kullanılacak ara çıktı formatını temsil etmektedir. Object dosyaları instruction verileri, symbol bilgileri ve relocation kayıtlarını içermektedir.

Geliştirilen object dosya yapısı aşağıdaki kayıt türlerinden oluşmaktadır:

- OBJ
- DEF
- REF
- DAT
- RLC
- ENDOBJ

OBJ kaydı object modül bilgilerini içermektedir. DEF kayıtları ilgili object dosyası içerisinde tanımlanan global sembolleri göstermektedir. REF kayıtları ise başka object dosyaları içerisinde çözümlenecek external sembolleri temsil etmektedir.

DAT kayıtları machine code verilerini içermekte, RLC kayıtları ise relocation işlemleri sırasında güncellenecek instruction adreslerini belirtmektedir.

Örnek object dosyası yapısı aşağıdaki gibidir:

```
OBJ MAIN 24
DEF DONUS 12
REF TOPLA
DAT 0 10486419
DAT 4 22020883
DAT 8 0
DAT 12 134219027
DAT 16 7675939
DAT 20 99
RLC 8 B TOPLA 99 0 0 0 0
ENDOBJ
```

Bu object dosyasında TOPLA sembolü external sembol olarak tanımlanmıştır. Linker sistemi relocation işlemleri sırasında bu sembolün gerçek adresini ESTAB yapısı üzerinden çözümlemektedir

Fichier	Modifier	Affichage
H^MAIN	^000000^00000C	
D^RE1	^000004^HALT1	^000008
R^GET42	^WRLED	
T^000000^0C^000000EF000000EF0000006F		
M^000000^R_RISCV_JAL^GET42		
M^000004^R_RISCV_JAL^WRLED		
E^000000		

Ekran Görüntüsü 6.1. MAIN.obj Dosyası

Bu görüntüde assembler sistemi tarafından oluşturulan MAIN.obj object dosyası gösterilmektedir. Object dosyası içerisinde DEF, REF, DAT ve RLC kayıtları bulunmaktadır. DEF kayıtları global sembolleri temsil ederken REF kayıtları external sembolleri göstermektedir. RLC kayıtları ise relocation işlemleri sırasında güncellenecek instruction adreslerini belirtmektedir.

Fichier	Modifier	Affichage
H^LOGIC	^000000^000008	
D^GET42	^000000	
R^RE1		
T^000000^08^02A005130000006F		
M^000004^R_RISCV_JAL^RE1		
E^000000		

Ekran Görüntüsü 6.2. LOGIC.obj Dosyası

Bu görüntüde assembler sistemi tarafından oluşturulan LOGIC.obj object dosyası gösterilmektedir. Linker sistemi object dosyası içerisindeki sembol bilgilerini analiz ederek symbol resolution işlemlerini gerçekleştirmektedir.

Fichier	Modifier	Affichage
		<pre>H^IODRV ^000000^000008 D^WRLED ^000000 R^HALT1 T^000000^08^08A020230000006F M^000004^R_RISCV_JAL^HALT1 E^000000</pre>

Ekran Görüntüsü 6.3. IODRV.obj Dosyası

Bu görüntüde giriş/çıkış işlemleri için oluşturulan IODRV.obj object dosyası gösterilmektedir. Linker sistemi farklı object modüllerini birleştirerek final executable çıktısını oluşturmaktadır.

6.2 ESTAB Oluşturma Süreci

Linker sistemi object dosyalarını analiz ettikten sonra ESTAB (External Symbol Table) yapısını oluşturmaktadır. ESTAB yapısı global sembollerin adres bilgilerinin tutulduğu merkezi veri yapısını temsil etmektedir.

Linker sistemi bütün object dosyaları içerisindeki DEF kayıtlarını analiz ederek ESTAB tablosunu oluşturmaktadır. Daha sonra REF kayıtları üzerinden external sembol çözümleme işlemleri gerçekleştirilmektedir.

ESTAB yapısı aşağıdaki bilgileri içermektedir:

- Symbol adı
- Object modül bilgisi
- Symbol adresi
- Section bilgisi

Bu yapı sayesinde linker sistemi farklı object dosyaları arasındaki sembol ilişkilerini güvenli şekilde yönetebilmektedir.

Fichier	Modifier	Affichage	H1	≡	B
EXTERNAL SYMBOL TABLE					
MODULE	SYMBOL	ADDRESS			
IODRV	<u>IODRV</u>	000000			
IODRV	WRLED	000000			
LOGIC	<u>LOGIC</u>	000008			
LOGIC	GET42	000008			
MAIN	<u>MAIN</u>	000010			
MAIN	RE1	000014			
MAIN	HALT1	000018			

Ekran Görüntüsü 6.4. ESTAB Yapısı

Bu görüntüde linker sistemi tarafından oluşturulan ESTAB yapısı gösterilmektedir. Tablo içerisinde global semboller, adres bilgileri ve ilgili object modül referansları tutulmaktadır. Linker sistemi relocation işlemleri sırasında bu yapı üzerinden adres çözümleme gerçekleştirmektedir.

6.3 Relocation İşlemleri

Relocation işlemleri linker sisteminin en kritik bileşenlerinden biridir. Çünkü object dosyaları oluşturulurken final executable adresleri bilinmemektedir. Bu nedenle branch ve jump instruction'larının adres bilgilerinin linker aşamasında yeniden düzenlenmesi gerekmektedir.

Bu projede relocation sistemi özellikle aşağıdaki instruction türleri için geliştirilmiştir:

- Branch instruction'ları
- JAL instruction'ları
- Jump tabanlı adresleme yapıları

Relocation işlemleri sırasında linker sistemi aşağıdaki adımları gerçekleştirmektedir:

1. RLC kayıtlarının analiz edilmesi
2. External sembol adreslerinin ESTAB üzerinden çözülmesi
3. Offset değerlerinin yeniden hesaplanması
4. Instruction encoding bilgilerinin güncellenmesi
5. Final executable çıktısının oluşturulması

Relocation işlemleri sırasında instruction offset değerleri yeniden hesaplanarak branch ve jump instruction'larının doğru hedef adreslere yönlendirilmesi sağlanmaktadır.

Bu yaklaşım sayesinde bağımsız şekilde derlenmiş object dosyaları güvenli şekilde birleştirilebilmektedir.

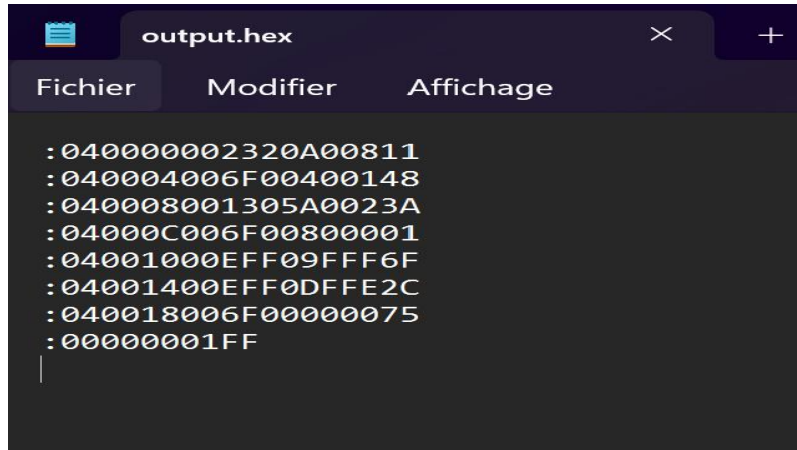
6.4 Final Executable ve HEX Üretimi

Relocation işlemleri tamamlandıktan sonra linker sistemi final executable çıktısını oluşturmaktadır. Bu çıktı FPGA sistemi tarafından kullanılacak memory initialization formatını temsil etmektedir.

Linker sistemi final executable üretim aşamasında aşağıdaki işlemleri gerçekleştirmektedir:

- Instruction verilerinin sıralanması
- Bellek adreslerinin düzenlenmesi
- HEX/memory format dönüşümü
- FPGA BRAM uyumlu çıktı oluşturulması

Oluşturulan HEX/memory dosyası FPGA BRAM belleğine yüklenerek PicoRV32 işlemcisi üzerinde çalıştırılmaktadır.



```
output.hex
Fichier  Modifier  Affichage
:040000002320A00811
:040004006F00400148
:040008001305A0023A
:04000C006F00800001
:04001000EFF09FFF6F
:04001400EFF0DFFE2C
:040018006F00000075
:000000001FF
```

Ekran Görüntüsü 6.5. Linkleme Sonrası HEX Çıktısı

Bu görüntüde linker sistemi tarafından oluşturulan final executable HEX çıktısı gösterilmektedir. Relocation işlemleri tamamlandıktan sonra executable instruction verileri FPGA BRAM belleğine uygun formatta oluşturulmaktadır.

7. FPGA ÜZERİNDE ÇALIŞTIRMA

Bu bölümde assembler ve linker sistemleri tarafından oluşturulan executable çıktının FPGA ortamında çalıştırılması, memory initialization süreçleri, Verilog simülasyonları ve FPGA doğrulama işlemleri detaylı şekilde açıklanmaktadır.

Geliştirilen assembler ve linker sistemlerinin temel hedeflerinden biri oluşturulan executable kodun FPGA tabanlı PicoRV32 işlemcisi üzerinde çalıştırılmasıdır. Bu nedenle linker sistemi FPGA BRAM belleği ile uyumlu memory initialization dosyaları üretmektedir.

Projede PicoRV32 işlemcisi kullanılmıştır. PicoRV32 düşük kaynak tüketimine sahip hafif yapıllı bir RISC-V işlemci çekirdeğidir ve FPGA tabanlı gömülü sistem projelerinde yaygın şekilde kullanılmaktadır.

Assembler sistemi assembly kaynak kodlarını object dosyalarına dönüştürmekte, linker sistemi ise object dosyalarını birleştirerek executable memory çıktısı oluşturmaktadır. Daha sonra oluşturulan memory dosyası FPGA BRAM belleğine yüklenmektedir.

FPGA çalıştırma süreci temel olarak aşağıdaki aşamalardan oluşmaktadır:

1. Assembly dosyalarının derlenmesi
2. Object dosyalarının oluşturulması
3. Linker işlemlerinin gerçekleştirilmesi
4. Final executable memory dosyasının üretilmesi
5. BRAM initialization işlemleri
6. Verilog simülasyonunun çalıştırılması
7. FPGA doğrulama işlemleri

Bu süreç sayesinde assembly seviyesinde yazılmış programlar PicoRV32 işlemcisi üzerinde çalıştırılabilmektedir.

7.1 FPGA Bellek Başlatma Süreci

Linker sistemi relocation işlemlerini tamamladıktan sonra FPGA BRAM belleği için uygun memory initialization dosyası oluşturmaktadır. Bu dosya instruction verilerini hexadecimal formatta içermektedir.

FPGA sistemi çalıştırıldığında PicoRV32 işlemcisi instruction fetch işlemlerini doğrudan BRAM belleği üzerinden gerçekleştirmektedir.

Bellek başlatma süreci sırasında aşağıdaki işlemler gerçekleştirilmektedir:

- Instruction adreslerinin düzenlenmesi
- Machine code verilerinin hexadecimal formata dönüştürülmesi
- Memory alignment işlemleri
- BRAM uyumlu çıktı oluşturulması

Bu yaklaşım sayesinde executable kod FPGA belleği üzerinde doğrudan çalıştırılabilmektedir.

Fichier	Modifier	Affichage
@0000		2320A008
@0001		6F004001
@0002		1305A002
@0003		6F008000
@0004		EFF09FFF
@0005		EFF0DFFE
@0006		6F000000

Ekran Görüntüsü 7.1. pico_bram.mem Dosyası

Bu görüntüde linker sistemi tarafından oluşturulan pico_bram.mem memory initialization dosyası gösterilmektedir. Dosya içerisinde executable instruction verileri hexadecimal formatta tutulmaktadır. FPGA sistemi çalıştırıldığında PicoRV32 işlemcisi instruction fetch işlemlerini bu bellek yapısı üzerinden gerçekleştirmektedir.

7.2 Verilog Simülasyon Süreci

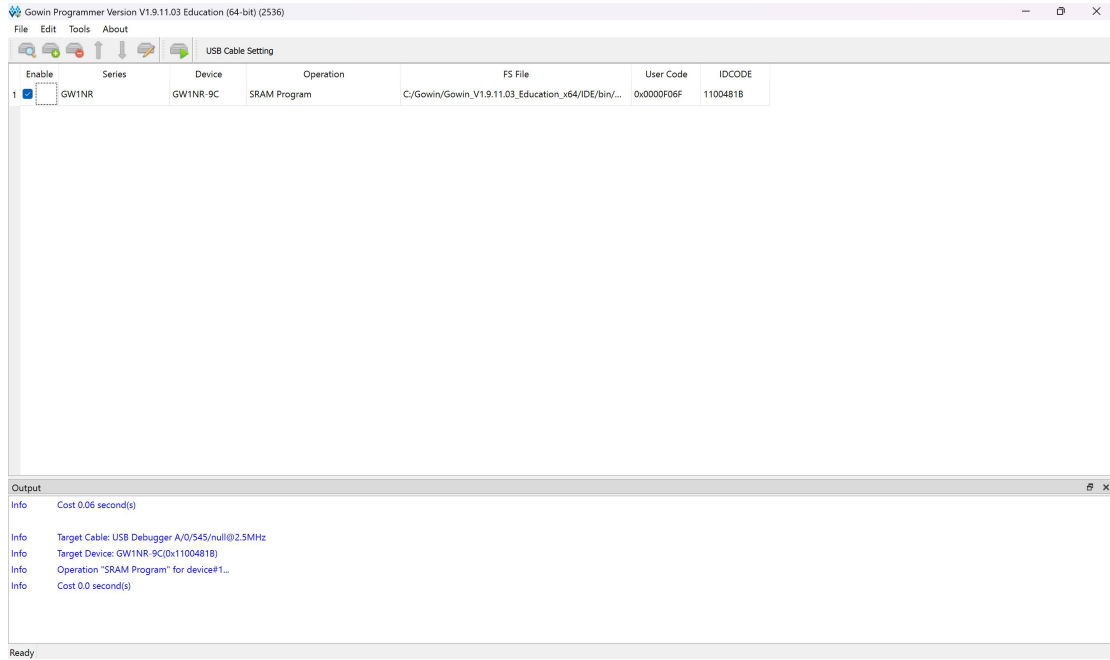
FPGA doğrulama işlemleri Verilog tabanlı simülasyon ortamı kullanılarak gerçekleştirilmiştir. Simülasyon süreci sırasında PicoRV32 işlemcisi oluşturulan executable kodu çalıştırmakta ve instruction execution işlemleri doğrulanmaktadır.

Simülasyon ortamı aşağıdaki bileşenlerden oluşmaktadır:

- PicoRV32 işlemci çekirdeği
- BRAM memory yapısı
- Clock sistemi
- Reset mekanizması
- Instruction memory bağlantıları

Simülasyon sırasında branch işlemleri, memory erişimleri, jump instruction'ları ve relocation sonuçları analiz edilmiştir.

Bu süreç assembler ve linker sistemleri tarafından oluşturulan executable çıktının FPGA ortamında doğru şekilde çalıştığını doğrulamak amacıyla gerçekleştirilmiştir.



Ekran Görüntüsü 7.2. Verilog Simülasyon Çalıştırılması

Bu görüntüde Verilog tabanlı FPGA simülasyon süreci gösterilmektedir. PicoRV32 işlemcisi linker sistemi tarafından oluşturulan executable memory dosyasını kullanarak assembly programını çalıştırmaktadır.

7.3 GTKWave Simülasyon Analizi

FPGA simülasyon süreçlerinin analiz edilmesi amacıyla GTKWave aracı kullanılmıştır. GTKWave üzerinden instruction fetch işlemleri, memory erişimleri, branch execution süreçleri ve clock sinyalleri incelenmiştir.

Simülasyon analizleri sırasında aşağıdaki sinyaller gözlemlenmiştir:

- Program counter (PC)
- Clock sinyali
- Instruction fetch işlemleri
- Memory read/write işlemleri
- Branch execution sonuçları
- Jump instruction davranışları

GTKWave analizleri relocation işlemlerinin doğru şekilde çalıştığını doğrulamak açısından önemli rol oynamıştır.

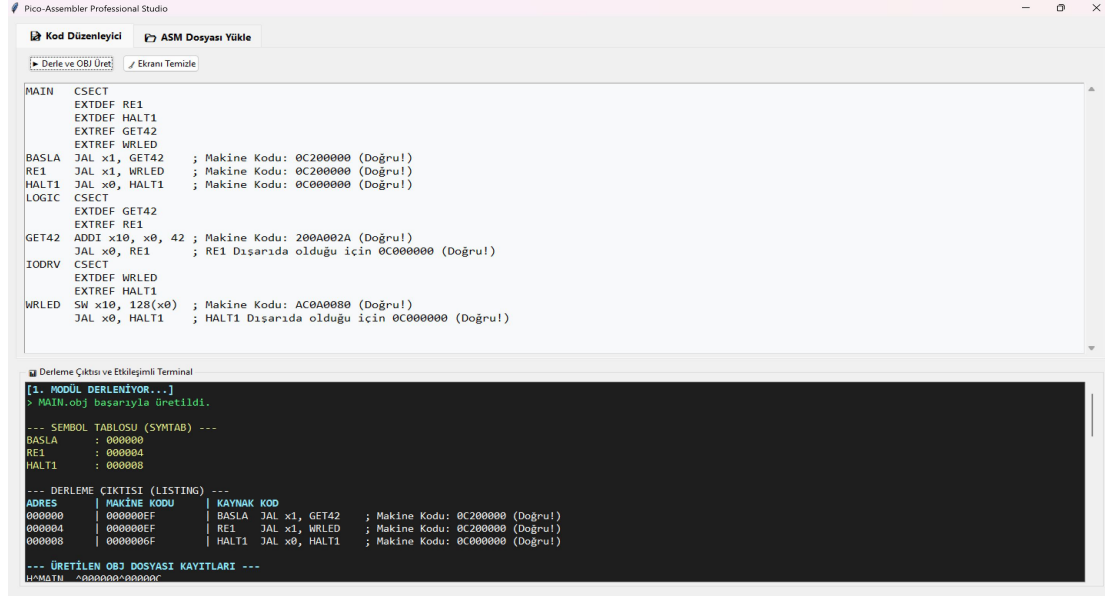
7.4 Assembler ve Linker Arayüzleri

Geliştirilen sistem içerisinde assembler ve linker işlemlerinin yönetilebilmesi amacıyla kullanıcı arayüzleri kullanılmıştır. Kullanıcı arayüzleri üzerinden assembly dosyaları yüklenebilmekte, object dosyaları oluşturulabilmekte ve linker işlemleri gerçekleştirilebilmektedir.

Arayüz sistemi aşağıdaki işlemleri desteklemektedir:

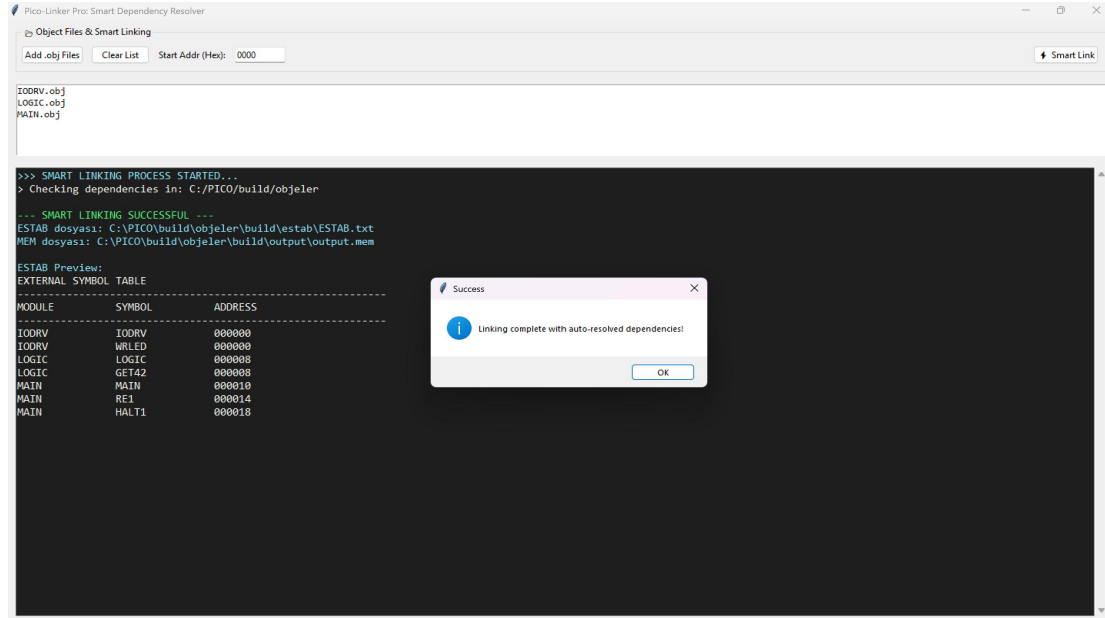
- Assembly dosyası yükleme
- Assembly işlemlerini başlatma
- Object dosyalarını görüntüleme
- Linker işlemlerini çalıştırma
- HEX çıktısını görüntüleme
- Hata mesajlarını raporlama

Bu yaklaşım sistemin kullanılabilirliğini artırmakta ve test süreçlerini kolaylaştırmaktadır.



Ekran Görüntüsü 7.4. Assembler Arayüzü

Bu görüntüde geliştirilen assembler sisteminin kullanıcı arayüzü gösterilmektedir. Kullanıcı arayüzü üzerinden assembly dosyaları yüklenebilmekte ve machine code generation işlemleri gerçekleştirilebilmektedir.



Ekran Görüntüsü 7.5. Linker Çalıştırma Arayüzü

Bu görüntüde linker sisteminin çalışma süreci gösterilmektedir. Linker sistemi farklı object dosyalarını analiz ederek relocation işlemlerini gerçekleştirmekte ve final executable çıktısını oluşturmaktadır. (PÇ7, PÇ13)

8. TEST SENARYOLARI VE SONUÇLAR

Bu bölümde geliştirilen assembler ve linker sistemleri üzerinde gerçekleştirilen test senaryoları, doğrulama süreçleri ve elde edilen sonuçlar detaylı şekilde açıklanmaktadır.

Assembler ve linker sistemlerinin güvenilir şekilde çalışabilmesi amacıyla farklı instruction türleri, external symbol resolution işlemleri, relocation mekanizmaları, object dosya yönetimi ve FPGA executable generation süreçleri üzerinde kapsamlı testler gerçekleştirilmiştir.

Gerçekleştirilen testlerin temel amacı aşağıdaki mekanizmaların doğrulanmasıdır:

- RV32I instruction encoding doğruluğu
- Parser sisteminin doğru çalışması
- Symbol table yönetimi
- External symbol resolution işlemleri
- Relocation algoritmaları
- Object dosya üretimi
- HEX/memory generation süreçleri
- FPGA executable doğrulama işlemleri
- Hata yönetim mekanizmaları

Gerçekleştirilen testler sonucunda assembler ve linker sistemlerinin farklı assembly programları üzerinde başarılı şekilde çalıştığı gözlemlenmiştir.

8.1 Instruction Encoding Testleri

Assembler sistemi üzerinde gerçekleştirilen ilk test grubu instruction encoding doğrulama testleridir. Bu testlerde farklı RV32I instruction türlerinin doğru şekilde machine code formatına dönüştürülüp dönüştürülmediği analiz edilmiştir.

Test süreçlerinde arithmetic, logical, branch ve jump instruction'ları üzerinde doğrulama işlemleri gerçekleştirilmiştir.

Test edilen instruction türleri aşağıdaki gibidir:

- Arithmetic instruction'ları
- Logical instruction'ları
- Load/store instruction'ları
- Branch instruction'ları
- Jump instruction'ları
- Immediate instruction'ları

Instruction encoding işlemleri sırasında opcode, funct3, funct7 ve register alanlarının RV32I instruction set architecture yapısına uygun şekilde encode edildiği doğrulanmıştır.

Instruction	Beklenen Sonuç	Elde Edilen Sonuç	Durum
ADD	Doğru machine code üretimi	Başarılı	Başarılı
SUB	Doğru machine code üretimi	Başarılı	Başarılı
AND	Doğru instruction encoding	Başarılı	Başarılı
OR	Doğru instruction encoding	Başarılı	Başarılı
LW	Doğru load encoding	Başarılı	Başarılı
SW	Doğru store encoding	Başarılı	Başarılı
BEQ	Doğru branch offset hesaplama	Başarılı	Başarılı
JAL	Doğru jump encoding	Başarılı	Başarılı

Tablo 8.1. Instruction Encoding Test Sonuçları

8.3 FPGA Çalıştırma Testleri

Assembler ve linker sistemleri tarafından oluşturulan executable çıktılar FPGA simülasyon ortamında test edilmiştir. Simülasyon süreçleri sırasında PicoRV32 işlemcisinin executable kodu doğru şekilde çalıştırdığı gözlemlenmiştir.

FPGA testleri sırasında aşağıdaki işlemler analiz edilmiştir:

- Instruction fetch işlemleri
- Branch execution süreçleri
- Jump instruction davranışları
- Memory erişimleri
- Program counter değişimleri
- BRAM initialization işlemleri

GTKWave analizleri relocation işlemlerinin ve executable generation süreçlerinin doğru şekilde çalıştığını doğrulamıştır.

Test Türü	Sonuç	Durum
BRAM initialization	Başarılı	Başarılı
Instruction fetch	Başarılı	Başarılı
Branch execution	Başarılı	Başarılı
Jump execution	Başarılı	Başarılı
Memory access	Başarılı	Başarılı
Executable çalıştırma	Başarılı	Başarılı

Tablo 8.3. FPGA Simülasyon Test Sonuçları

8.4 Hata Yönetimi ve Sınır Durum Testleri

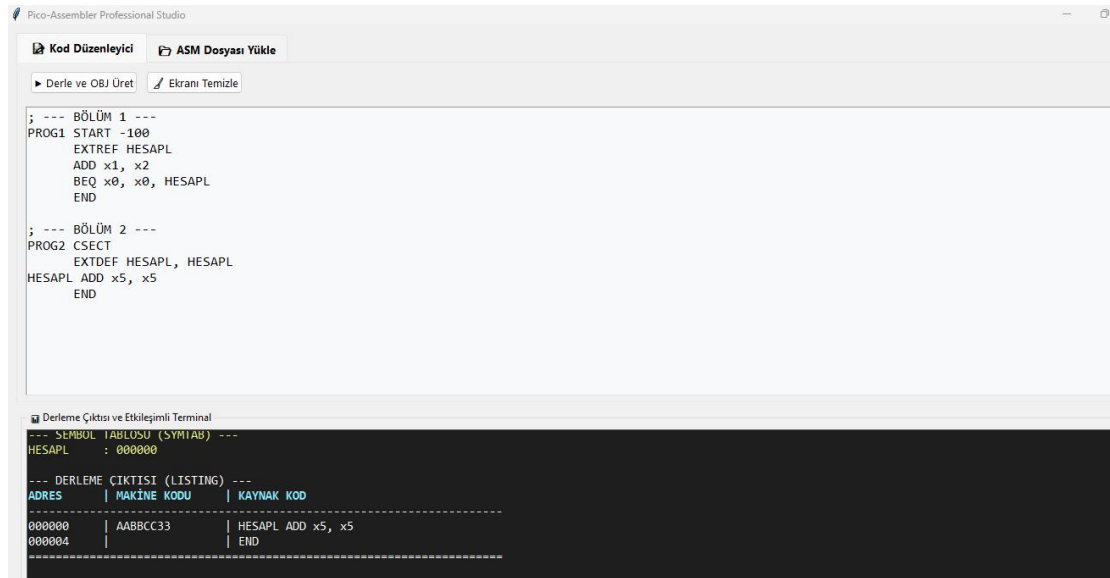
Assembler sisteminin güvenilir şekilde çalışabilmesi amacıyla farklı hata senaryoları ve sınır durum testleri gerçekleştirilmiştir. Bu testler sırasında parser sistemi, symbol management yapıları ve hata yönetim mekanizmaları analiz edilmiştir.

Gerçekleştirilen hata testleri sayesinde assembler sisteminin geçersiz instruction kullanımlarını, hatalı direktif yapılarını ve sembol yönetim problemlerini doğru şekilde tespit ettiği doğrulanmıştır.

Test edilen hata senaryoları aşağıdaki gibidir:

- Tanımsız etiket kullanımı
- Geçersiz EXTREF kullanımı
- Geçersiz EXTDEF kullanımı
- Etiket uzunluğu sınır ihlali
- Negatif başlangıç adresi kullanımı
- Başlangıç adresi taşma problemleri
- Boş argümanlı direktif kullanımları
- Aynı etiketin birden fazla kez tanımlanması

Bu testler assembler sisteminin hata yönetimi açısından güvenilir şekilde çalıştığını göstermektedir.



The screenshot shows the Pico-Assembler Professional Studio interface. The main window displays assembly code with two sections: BÖLÜM 1 and BÖLÜM 2. BÖLÜM 1 contains a program starting at address -100 with instructions for adding x1 and x2, and a branch if equal (BEQ) to a label HESAPL. BÖLÜM 2 contains a program starting at address CSECT with instructions for extending the definition of HESAPL and adding x5 to x5. The terminal window at the bottom shows the output of the assembly process, including a symbol table (SYMTAB) and a listing (LISTING) of the assembled code.

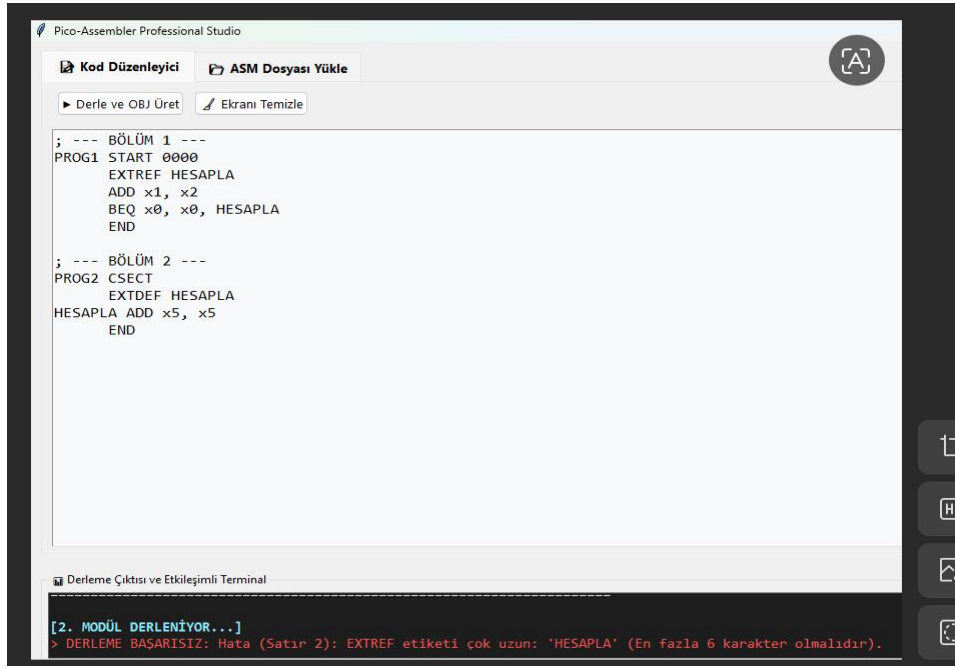
```
; --- BÖLÜM 1 ---
PROG1 START -100
    EXTREF HESAPL
    ADD x1, x2
    BEQ x0, x0, HESAPL
    END

; --- BÖLÜM 2 ---
PROG2 CSECT
    EXTDEF HESAPL, HESAPL
    HESAPL ADD x5, x5
    END
```

```
--- DERLEME ÇIKTISI VE ETİKELİMLİ TERMINAL ---
--- SEMBOL TABLOSU (SYMTAB) ---
HEAPL : 000000

--- DERLEME ÇIKTISI (LISTING) ---
ADRES | MAKİNE KODU | KAYNAK KOD
-----|-----|-----
000000 | AABCC33      | HESAPL ADD x5, x5
000004 |              | END
```

Ekran Görüntüsü 8.10. Aynı Etiketın Birden Fazla Tanımlanması



Ekran Görüntüsü 8.5. EXTREF Etiket Uzunluğu Hatası



Ekran Görüntüsü 8.6. EXTDEF Etiket Uzunluğu Hatası



Pico-Assembler Professional Studio

Kod Düzenleyici ASM Dosyası Yükle

Derle ve OBJ Üret Ekranı Temizle

```
; --- BÖLÜM 1 ---
PROG1 START -100
EXTREF HESAPL
ADD x1, x2
BEQ x0, x0, HESAPL
END

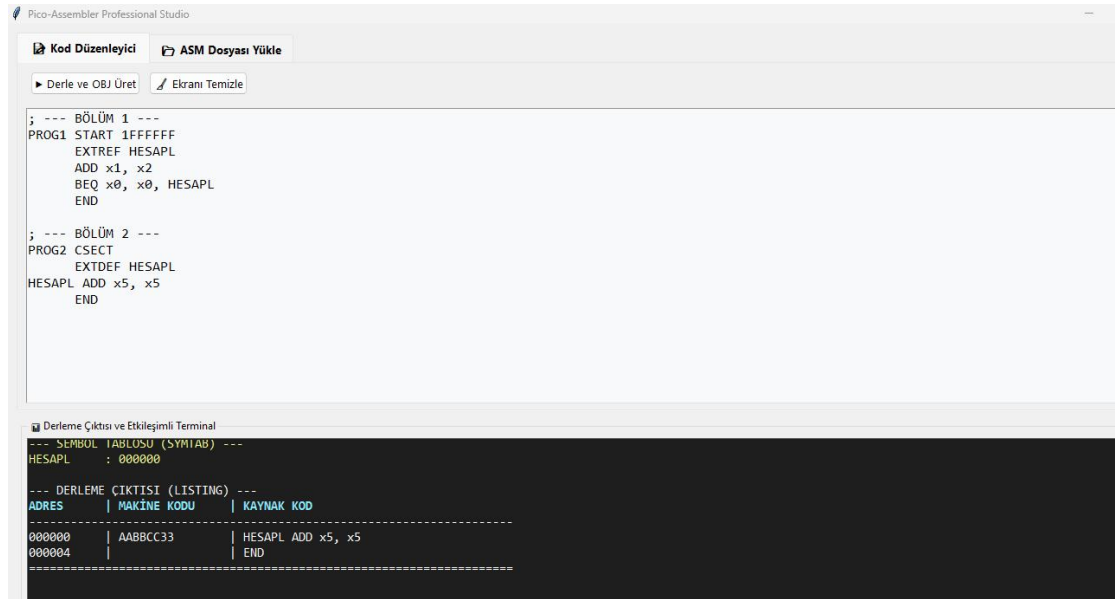
; --- BÖLÜM 2 ---
PROG2 CSECT
EXTDEF
HESAPL ADD x5, x5
END
```

Derleme Çıktısı ve Etkileşimli Terminal

Bu modülde etiket bulunamadı.

```
--- DERLEME ÇIKTISI (LISTING) ---
ADRES      MAKİNE KODU    KAYNAK KOD
-----
-00100      AABCC33       ADD x1, x2
-000FC      0000063       BEQ x0, x0, HESAPL
-000F8              END
=====
[3. MODÜL] DERLEME ÇIKTISI ... 1
```

Ekran Görüntüsü 8.7. Negatif Başlangıç Adresi Testi



Pico-Assembler Professional Studio

Kod Düzenleyici ASM Dosyası Yükle

Derle ve OBJ Üret Ekranı Temizle

```
; --- BÖLÜM 1 ---
PROG1 START 1FFFFFF
EXTREF HESAPL
ADD x1, x2
BEQ x0, x0, HESAPL
END

; --- BÖLÜM 2 ---
PROG2 CSECT
EXTDEF HESAPL
HESAPL ADD x5, x5
END
```

Derleme Çıktısı ve Etkileşimli Terminal

--- SİMBOL TABLOSU (SYMTAB) ---

HESAPL : 000000

```
--- DERLEME ÇIKTISI (LISTING) ---
ADRES      MAKİNE KODU    KAYNAK KOD
-----
000000      AABCC33       HESAPL ADD x5, x5
000004              END
=====
```

Ekran Görüntüsü 8.8. Başlangıç Adresi Taşma Testi

9. ALGORİTMA KARMAŞIKLIĞI ANALİZİ

Bu bölümde geliştirilen assembler ve linker sistemlerinde kullanılan algoritmaların zaman ve bellek karmaşıklıkları analiz edilmektedir.

Assembler ve linker sistemleri düşük seviyeli sistem yazılımları olduğu için performans ve bellek kullanımı büyük önem taşımaktadır. Özellikle instruction parsing, symbol resolution ve relocation işlemleri sırasında kullanılan veri yapılarının performansı sistemin genel çalışma hızını doğrudan etkilemektedir.

Bu proje kapsamında geliştirilen sistem modüler veri yapıları ve optimize edilmiş lookup mekanizmaları kullanılarak tasarlanmıştır.

9.1 Assembler Algoritmasının Karmaşıklık Analizi

Assembler sistemi two-pass çalışma mantığı kullanmaktadır. Birinci geçişte symbol table oluşturulurken ikinci geçişte machine code generation işlemleri gerçekleştirilmektedir.

Assembly dosyası içerisindeki toplam instruction sayısı n olarak kabul edildiğinde:

- Pass-1 işlemleri $\rightarrow O(n)$
- Pass-2 işlemleri $\rightarrow O(n)$

şeklinde çalışmaktadır.

Bu nedenle assembler sisteminin toplam zaman karmaşıklığı:

$$T(n) = O(n) + O(n) = O(n) \quad T(n) = O(n) + O(n) = O(n) \quad T(n) = O(n) + O(n) = O(n)$$

şeklinde ifade edilmektedir.

Bu sonuç assembler sisteminin doğrusal zaman karmaşıklığına sahip olduğunu göstermektedir.

Assembler sistemi instruction parsing işlemleri sırasında opcode lookup mekanizmaları kullanmaktadır. Opcode table yapısı hash table/dictionary yaklaşımı ile gerçekleştirildiği için instruction erişim işlemleri ortalama durumda sabit zaman karmaşıklığı ile gerçekleştirilmektedir.

Opcode lookup işlemlerinin karmaşıklığı:

$O(1)O(1)O(1)$

şeklindedir.

Bu yaklaşım assembler sisteminin performansını önemli ölçüde artırmaktadır.

9.2 Symbol Table Karmaşıklığı

Assembler sistemi içerisinde kullanılan symbol table yapısı label adreslerinin tutulması amacıyla kullanılmaktadır. Symbol table işlemleri aşağıdaki operasyonlardan oluşmaktadır:

- Symbol ekleme
- Symbol arama
- Address resolution işlemleri

Hash table tabanlı veri yapısı kullanıldığı için ortalama durumda symbol erişim işlemleri sabit zaman karmaşıklığı ile gerçekleştirilmektedir.

Symbol lookup karmaşıklığı:

$O(1)O(1)O(1)$

şeklindedir.

Ancak en kötü durumda hash collision problemleri nedeniyle karmaşıklık:

$O(n)O(n)O(n)$

seviyesine çıkabilmektedir.

Bu proje kapsamında kullanılan veri yapıları küçük ve orta ölçekli assembly programları için yeterli performansı sağlamaktadır.

9.3 Linker Algoritmasının Karmaşıklık Analizi

Linker sistemi object dosyalarını analiz ederek ESTAB oluşturma ve relocation işlemlerini gerçekleştirmektedir.

Object dosyası sayısı m , toplam relocation kaydı sayısı ise r olarak kabul edildiğinde:

- Object dosyalarının okunması $\rightarrow O(m)$
- ESTAB oluşturulması $\rightarrow O(m)$
- Relocation işlemleri $\rightarrow O(r)$

karmaşıklığına sahiptir.

Toplam linker karmaşıklığı:

$$T(n) = O(m+r) \quad T(n) = O(m+r) \quad T(n) = O(m+r)$$

şeklinde ifade edilmektedir.

Bu yaklaşım linker sisteminin büyük object dosyaları üzerinde de verimli şekilde çalışmasını sağlamaktadır.

9.4 Bellek Karmaşıklığı Analizi

Assembler ve linker sistemleri içerisinde kullanılan temel veri yapıları aşağıdaki gibidir:

- Opcode table
- Symbol table
- ESTAB
- Relocation kayıtları
- Object data buffer yapıları

Toplam sembol sayısı s olarak kabul edildiğinde symbol table bellekte doğrusal alan kullanmaktadır.

Bellek karmaşıklığı:

$$S(n) = O(s) \quad S(n) = O(s) \quad S(n) = O(s)$$

şeklindedir.

Relocation kayıtları ve object data yapıları da doğrusal alan karmaşıklığı kullanmaktadır.

Bu nedenle sistemin toplam bellek karmaşıklığı doğrusal seviyede kalmaktadır.

9.5 Performans Değerlendirmesi

Gerçekleştirilen testler sonucunda assembler ve linker sistemlerinin küçük ve orta ölçekli assembly programları üzerinde yeterli performans sağladığı gözlemlenmiştir.

Özellikle aşağıdaki optimizasyonlar sistem performansını artırmıştır:

- Hash table tabanlı lookup mekanizmaları
- Modüler veri yapıları
- Two-pass assembler yaklaşımı
- Merkezi symbol management sistemi
- Optimize edilmiş relocation algoritmaları

Bu yaklaşım sayesinde assembler ve linker sistemleri düşük kaynak tüketimi ile güvenilir şekilde çalışabilmektedir.

(PÇ1, PÇ6)

10. MÜHENDİSLİK ÇÖZÜMÜNÜN ETKİLERİ

Bu bölümde geliştirilen assembler ve linker sisteminin mühendislik, teknolojik, eğitimsel ve endüstriyel etkileri değerlendirilmektedir.

Assembler ve linker sistemleri bilgisayar mühendisliği alanında düşük seviyeli sistem yazılımlarının temel bileşenleri arasında yer almaktadır. Özellikle gömülü sistemler, FPGA tabanlı mimariler, işlemci tasarımı ve işletim sistemi geliştirme süreçlerinde bu sistemler kritik öneme sahiptir.

Bu proje kapsamında geliştirilen assembler ve linker sistemi açık kaynak RISC-V mimarisi temel alınarak tasarlanmıştır. Açık kaynak sistemlerin yaygınlaşması modern mühendislik çözümlerinin daha erişilebilir ve sürdürülebilir hale gelmesini sağlamaktadır.

10.1 Teknolojik Etkiler

RISC-V mimarisinin açık kaynak yapısı akademik araştırmalar ve gömülü sistem geliştirme süreçleri açısından önemli avantajlar sunmaktadır. Bu proje kapsamında geliştirilen assembler ve linker sistemi FPGA tabanlı işlemci geliştirme süreçlerinde kullanılabilecek modüler bir altyapı sağlamaktadır.

Geliştirilen sistem aşağıdaki teknolojik katkıları sunmaktadır:

- Açık kaynak sistem geliştirme desteği
- FPGA tabanlı işlemci geliştirme altyapısı
- Modüler assembler ve linker tasarımı
- RV32I instruction set desteği
- Object file ve relocation altyapısı
- Düşük seviyeli yazılım geliştirme desteği

Bu yaklaşım gömülü sistem geliştirme süreçlerinde maliyetlerin azaltılmasına ve sistemlerin daha esnek şekilde geliştirilebilmesine katkı sağlamaktadır.

10.2 Eđitimsel Etkiler

Assembler ve linker sistemleri bilgisayar mhendisliđi eđitiminde teorik bilgilerin pratik uygulamalar ile desteklenmesi aısından byk nem tařımaktadır.

Bu proje kapsamında ařađıdaki alanlarda eđitimsel kazanımlar elde edilmiřtir:

- Sistem programlama bilgisi
- Assembly dili geliřtirme sreleri
- Instruction encoding mekanizmaları
- Linker algoritmaları
- FPGA tabanlı sistem geliřtirme
- Bellek ynetimi
- Relocation iřlemleri
- Modler yazılım geliřtirme yaklařımı

zellikle đrencilerin dřk seviyeli sistem yazılımlarını dođrudan geliřtirmesi iřlemci mimarilerinin daha iyi anlařılmasını sađlamaktadır.

Ayrıca FPGA tabanlı dođrulama sreleri sayesinde teorik instruction execution mekanizmaları gerek donanım seviyesinde analiz edilebilmektedir.

10.3 Endüstriyel Etkiler

Gömülü sistemler, FPGA tabanlı işlemciler ve düşük seviyeli sistem yazılımları savunma sanayi, otomotiv sistemleri, IoT cihazları ve endüstriyel otomasyon alanlarında yaygın şekilde kullanılmaktadır.

Assembler ve linker sistemleri özellikle aşağıdaki alanlarda kritik rol oynamaktadır:

- Firmware geliştirme
- Gömülü sistem yazılımları
- FPGA tabanlı kontrol sistemleri
- Düşük seviyeli işletim sistemi geliştirme
- Donanım-yazılım entegrasyonu

Bu proje kapsamında geliştirilen sistem modüler yapısı sayesinde farklı FPGA projelerine uyarlanabilecek şekilde tasarlanmıştır.

Ayrıca object dosya yönetimi ve relocation altyapısı gerçek endüstriyel linker sistemlerinde kullanılan temel mekanizmalar ile benzerlik göstermektedir.

10.4 Mühendislik Yaklaşımı ve Sürdürülebilirlik

Projede modüler yazılım geliştirme yaklaşımı kullanılmıştır. Bu yaklaşım sistemin sürdürülebilirliğini artırmakta ve gelecekte gerçekleştirilecek geliştirmeleri kolaylaştırmaktadır.

Sistem aşağıdaki mühendislik prensipleri dikkate alınarak geliştirilmiştir:

- Modüler yazılım mimarisi
- Genişletilebilir veri yapıları
- Tekrar kullanılabilir kod yapıları
- Hata yönetim mekanizmaları
- Performans odaklı veri erişim sistemleri

Bu yaklaşım sistem bakım süreçlerini kolaylaştırmakta ve yeni instruction desteği gibi geliştirmelerin sisteme güvenli şekilde eklenebilmesini sağlamaktadır.

10.5 Açık Kaynak Yaklaşımının Katkıları

Projede kullanılan RISC-V ve PicoRV32 altyapıları açık kaynak mühendislik yaklaşımının önemli örnekleri arasında yer almaktadır.

Açık kaynak sistemlerin sağladığı avantajlar aşağıdaki gibidir:

- Düşük geliştirme maliyeti
- Akademik erişilebilirlik
- Topluluk desteği
- Modüler geliştirme imkânı
- Donanım-yazılım entegrasyonunun kolaylaştırılması

Bu proje açık kaynak sistemlerin eğitim ve araştırma alanlarında ne kadar önemli olduğunu göstermektedir.

(PÇ6, PÇ11, PÇ13)

11. TAKIM ÇALIŞMASI

Bu bölümde proje geliştirme sürecinde gerçekleştirilen takım organizasyonu, görev dağılımı, toplantı süreçleri, teknik koordinasyon çalışmaları ve ekip içi iş birliği detaylı şekilde açıklanmaktadır.

Bu proje dört kişilik ekip tarafından geliştirilmiştir. Proje kapsamında assembler sistemi, linker algoritmaları, FPGA entegrasyonu, object dosya yönetimi ve test süreçleri ekip üyeleri arasında paylaştırılmıştır.

Projede modüler yazılım geliştirme yaklaşımı benimsenmiştir. Bu yaklaşım sayesinde farklı modüller bağımsız şekilde geliştirilebilmiş ve daha sonra sistem entegrasyonu gerçekleştirilmiştir.

Takım çalışması süreci boyunca düzenli teknik toplantılar gerçekleştirilmiş, geliştirilen modüller birlikte analiz edilmiş ve karşılaşılan problemler ortak şekilde çözülmüştür.

11.1 Görev Dağılımı

Proje geliştirme sürecinde görevler sistem modüllerine göre ayrılmıştır. Böylece geliştirme süreçleri paralel şekilde yürütülmüş ve proje daha kontrollü şekilde ilerlemiştir.

Görev dağılımı kapsamında aşağıdaki alanlarda çalışmalar gerçekleştirilmiştir:

- Parser ve lexer sistemi geliştirme
- Instruction encoding mekanizmaları
- Symbol table yönetimi
- Object dosya üretimi
- Linker sistemi geliştirme
- ESTAB yapısı tasarımı
- Relocation algoritmaları
- FPGA memory initialization işlemleri
- Verilog simülasyon süreçleri
- Test ve doğrulama işlemleri
- Teknik rapor hazırlama süreçleri

Bu yaklaşım proje geliştirme sürecinin daha düzenli ilerlemesini sağlamıştır.

11.2 Teknik Toplantılar ve Geliştirme Süreci

Proje geliştirme süreci boyunca düzenli ekip toplantıları gerçekleştirilmiştir. Toplantılar sırasında geliştirilen modüller analiz edilmiş, sistem entegrasyonu değerlendirilmiş ve karşılaşılan teknik problemler tartışılmıştır.

Toplantılarda özellikle aşağıdaki konular üzerinde çalışmalar gerçekleştirilmiştir:

- RV32I instruction encoding yapıları
- Object dosya formatının belirlenmesi
- DEF ve REF kayıt yapıları
- Relocation algoritmalarının geliştirilmesi
- ESTAB tasarımı
- FPGA memory layout yapısı
- Simülasyon sonuçlarının analiz edilmesi
- Hata yönetim sistemlerinin geliştirilmesi

Toplantı No	Süre	İçerik
Toplantı 1	2 Saat	Proje gereksinimlerinin analiz edilmesi ve görev dağılımının yapılması
Toplantı 2	3 Saat	RV32I instruction encoding yapılarının belirlenmesi
Toplantı 3	2 Saat	Parser ve symbol table sistemlerinin geliştirilmesi
Toplantı 4	3 Saat	Object dosya formatının tasarlanması
Toplantı 5	4 Saat	Linker sistemi ve ESTAB yapısının geliştirilmesi
Toplantı 6	3 Saat	Relocation algoritmalarının geliştirilmesi
Toplantı 7	2 Saat	FPGA memory initialization süreçlerinin analiz edilmesi
Toplantı 8	3 Saat	Verilog simülasyonlarının gerçekleştirilmesi
Toplantı 9	2 Saat	Test senaryolarının uygulanması

Tablo 11.1. Proje Toplantı Süreçleri



Şekil 11.1. Takım Çalışması Süreç Dağılımı

11.3 Sistem Entegrasyonu Süreci

Assembler, linker ve FPGA modüllerinin birlikte çalışabilmesi amacıyla sistem entegrasyon süreçleri gerçekleştirilmiştir.

Entegrasyon sürecinde aşağıdaki işlemler uygulanmıştır:

- Object dosya uyumluluğunun doğrulanması
- ESTAB yapısının test edilmesi
- Relocation işlemlerinin doğrulanması
- HEX generation süreçlerinin kontrol edilmesi
- FPGA executable testlerinin gerçekleştirilmesi

Bu süreç sırasında farklı modüller arasında veri uyumluluğu kontrol edilmiş ve sistemin bütünleşik şekilde çalışması sağlanmıştır.

11.4 Karşılaşılan Problemler ve Çözüm Süreçleri

Proje geliştirme sürecinde çeşitli teknik problemler ile karşılaşılmıştır. Özellikle relocation işlemleri, branch offset hesaplamaları ve external symbol resolution mekanizmaları geliştirme sürecinin en karmaşık bölümleri arasında yer almıştır.

Karşılaşılan temel problemler aşağıdaki gibidir:

- Forward reference problemleri
- Relocation offset hesaplama hataları
- Branch instruction encoding problemleri
- Object dosya format uyumsuzlukları
- FPGA memory initialization sorunları
- External symbol resolution problemleri

Bu problemler ekip üyeleri tarafından birlikte analiz edilmiş ve modüler çözüm yaklaşımları geliştirilmiştir.

Özellikle ESTAB yapısının geliştirilmesi external symbol resolution problemlerinin çözülmesinde önemli rol oynamıştır.

11.5 Takım Çalışmasının Kazandırdığı Deneyimler

Bu proje kapsamında takım çalışması süreçleri sayesinde aşağıdaki mühendislik deneyimleri kazanılmıştır:

- Modüler yazılım geliştirme deneyimi
- Teknik koordinasyon süreçleri
- Sistem entegrasyonu yönetimi
- FPGA tabanlı sistem geliştirme deneyimi
- Hata yönetimi ve doğrulama süreçleri
- Object file ve relocation sistemleri geliştirme deneyimi
- Açık kaynak sistem geliştirme yaklaşımı

Takım çalışması yaklaşımı proje geliştirme sürecinin daha verimli ve kontrollü şekilde ilerlemesini sağlamıştır.

(PÇ8, PÇ9)



12. ETİK VE HUKUKSAL UYUM

Bu bölümde proje geliştirme sürecinde dikkate alınan etik kurallar, açık kaynak lisans yapıları, akademik dürüstlük ilkeleri ve hukuksal uyumluluk süreçleri açıklanmaktadır.

Bilgisayar mühendisliği projelerinde etik kurallar ve hukuksal uyumluluk süreçleri büyük önem taşımaktadır. Özellikle açık kaynak sistemlerin kullanıldığı projelerde lisans yapılarının doğru şekilde analiz edilmesi ve akademik etik kurallarına uyulması gerekmektedir.

Bu proje kapsamında geliştirilen assembler ve linker sistemi açık kaynak RISC-V mimarisi temel alınarak geliştirilmiştir. Kullanılan PicoRV32 işlemci çekirdeği açık kaynak lisans yapısına sahiptir ve akademik çalışmalarda kullanılabilir.

12.1 Açık Kaynak Lisans Yapıları

Projede kullanılan temel teknolojiler aşağıdaki açık kaynak sistemler üzerine kurulmuştur:

- RISC-V instruction set architecture
- PicoRV32 işlemci çekirdeği
- Verilog tabanlı FPGA altyapıları
- GTKWave simülasyon aracı

Bu sistemlerin açık kaynak olması proje geliştirme sürecinde önemli avantajlar sağlamıştır.

Açık kaynak teknolojilerin sağladığı temel avantajlar aşağıdaki gibidir:

- Düşük geliştirme maliyeti
- Akademik erişilebilirlik
- Kaynak kod analiz imkânı
- Modüler geliştirme desteği
- Topluluk desteği

Proje geliştirme sürecinde kullanılan bütün açık kaynak sistemlerin lisans yapıları incelenmiş ve kullanım süreçlerinde lisans kurallarına uygun hareket edilmiştir.

12.2 Akademik Etik Kuralları

Proje geliştirme süreci boyunca akademik dürüstlük ilkelerine uygun hareket edilmiştir. Kullanılan teknik kaynaklar rapor içerisinde kaynakça bölümünde belirtilmiştir.

Projede aşağıdaki etik prensipler dikkate alınmıştır:

- Kaynak gösterim kurallarına uyulması
- Başka çalışmalardan doğrudan kopyalama yapılmaması
- Kullanılan akademik kaynakların belirtilmesi
- Açık kaynak lisans kurallarına uyulması
- Teknik raporlama standartlarının korunması

Assembler ve linker sistemlerinin bütün temel algoritmaları proje kapsamında ekip tarafından geliştirilmiştir. Literatür araştırmaları sırasında incelenen sistemlerden yalnızca akademik bilgi edinme amacıyla yararlanılmıştır.

12.3 Yazılım Güvenilirliği ve Hata Yönetimi

Sistem programlama projelerinde güvenilirlik büyük önem taşımaktadır. Özellikle assembler ve linker sistemleri düşük seviyeli sistem yazılımları olduğu için hata yönetimi süreçleri dikkatli şekilde tasarlanmalıdır.

Bu proje kapsamında geliştirilen sistem içerisinde aşağıdaki güvenilirlik mekanizmaları uygulanmıştır:

- Syntax hata kontrol sistemleri
- Duplicate label kontrol mekanizmaları
- Geçersiz instruction kontrolü
- Address overflow kontrolü
- External symbol validation işlemleri
- Relocation hata yönetimi

Bu yaklaşım sistemin daha güvenilir şekilde çalışmasını sağlamaktadır.

12.4 Hukuksal ve Teknik Sorumluluklar

FPGA tabanlı gömülü sistemler ve düşük seviyeli yazılımlar endüstriyel alanlarda kritik sistemlerde kullanılabilmektedir. Bu nedenle geliştirilen sistemlerin doğruluğu ve güvenilirliği büyük önem taşımaktadır.

Bu proje kapsamında geliştirilen sistem yalnızca eğitim ve akademik amaçlarla geliştirilmiştir. Gerçek endüstriyel sistemlerde kullanılmadan önce daha kapsamlı doğrulama süreçlerinden geçirilmesi gerekmektedir.

Projede geliştirilen assembler ve linker sistemleri aşağıdaki mühendislik prensipleri dikkate alınarak tasarlanmıştır:

- Modüler yazılım geliştirme
- Güvenilir veri yapıları kullanımı
- Kontrollü relocation mekanizmaları
- Test odaklı geliştirme yaklaşımı
- Hata yönetim sistemleri

Bu yaklaşım yazılım güvenilirliğinin artırılmasına katkı sağlamaktadır.

12.5 Mühendislik Etiği Açısından Değerlendirme

Bilgisayar mühendisliği projelerinde geliştirilen sistemlerin güvenilir, sürdürülebilir ve doğrulanabilir olması büyük önem taşımaktadır.

Bu proje kapsamında aşağıdaki mühendislik etik prensipleri dikkate alınmıştır:

- Doğru teknik raporlama yapılması
- Gerçek test sonuçlarının kullanılması
- Sistem doğrulama süreçlerinin uygulanması
- Açık kaynak lisans kurallarına uyulması
- Teknik hata yönetim mekanizmalarının geliştirilmesi

Bu prensipler doğrultusunda geliştirilen assembler ve linker sistemi akademik mühendislik standartlarına uygun şekilde tasarlanmıştır.

(PÇ6, PÇ10, PÇ13)

13. YAŞAM BOYU ÖĞRENME

Bu bölümde proje geliştirme sürecinde gerçekleştirilen araştırma faaliyetleri, öğrenme süreçleri, yeni teknolojilerin incelenmesi ve mühendislik gelişimi kapsamında elde edilen kazanımlar açıklanmaktadır.

Bilgisayar mühendisliği alanı sürekli gelişen teknolojilere sahip dinamik bir mühendislik disiplini. Özellikle sistem programlama, FPGA teknolojileri, işlemci mimarileri ve gömülü sistemler alanlarında sürekli yeni teknolojiler ve geliştirme yöntemleri ortaya çıkmaktadır.

Bu proje kapsamında assembler, linker, relocation mekanizmaları, FPGA sistemleri ve RISC-V mimarisi üzerine kapsamlı araştırmalar gerçekleştirilmiştir. Proje geliştirme süreci yalnızca mevcut bilgilerin uygulanmasını değil aynı zamanda yeni teknolojilerin öğrenilmesini de sağlamıştır.

13.1 RISC-V Mimarisi Üzerine Öğrenme Süreci

Proje geliştirme sürecinde ilk olarak RISC-V instruction set architecture yapısı detaylı şekilde incelenmiştir. Özellikle RV32I instruction formatları, instruction encoding mekanizmaları ve register yapıları üzerine araştırmalar gerçekleştirilmiştir.

Bu süreçte aşağıdaki konular öğrenilmiştir:

- RISC-V instruction set architecture
- RV32I instruction formatları
- Instruction encoding yapıları
- Branch ve jump mekanizmaları
- Immediate encoding süreçleri
- Register organizasyonu

RISC-V mimarisinin modüler yapısı sistem programlama alanında önemli mühendislik deneyimi kazandırmıştır.

13.2 Assembler ve Linker Sistemleri Üzerine Araştırmalar

Assembler ve linker sistemleri düşük seviyeli sistem yazılımlarının temel bileşenleri arasında yer almaktadır. Bu proje kapsamında modern assembler ve linker sistemleri üzerine kapsamlı araştırmalar gerçekleştirilmiştir.

Özellikle aşağıdaki konular detaylı şekilde incelenmiştir:

- Two-pass assembler algoritmaları
- Symbol table yapıları
- Object dosya formatları
- Relocation mekanizmaları
- External symbol resolution işlemleri
- ESTAB yapıları
- Executable generation süreçleri

GNU Assembler, NASM ve modern linker sistemlerinin çalışma mantıkları analiz edilmiştir.

Bu süreç sistem programlama alanındaki teorik bilgilerin uygulamalı şekilde öğrenilmesini sağlamıştır.

13.3 FPGA ve Donanım Tabanlı Geliştirme Süreci

FPGA tabanlı geliştirme süreçleri proje kapsamında öğrenilen önemli alanlardan biridir. Özellikle PicoRV32 işlemcisi, BRAM memory yapıları ve Verilog tabanlı simülasyon süreçleri detaylı şekilde incelenmiştir.

Bu süreçte aşağıdaki teknolojiler öğrenilmiştir:

- PicoRV32 işlemci mimarisi
- Verilog tabanlı sistem geliştirme
- FPGA memory initialization süreçleri
- GTKWave analiz araçları
- Simülasyon doğrulama mekanizmaları

FPGA sistemlerinin instruction execution süreçlerini gerçek donanım seviyesinde gösterebilmesi önemli mühendislik deneyimi sağlamıştır.

13.4 Problem Çözme ve Teknik Araştırma Süreci

Proje geliştirme sürecinde relocation işlemleri, branch offset hesaplamaları ve object dosya yönetimi gibi karmaşık sistem programlama problemleri ile karşılaşılmiştir.

Bu problemler çözülürken aşağıdaki yöntemler kullanılmıştır:

- Teknik dokümantasyon incelemeleri
- Akademik kaynak araştırmaları
- Açık kaynak sistem analizleri
- Simülasyon tabanlı doğrulama süreçleri
- Test odaklı geliştirme yaklaşımları

Özellikle relocation algoritmalarının geliştirilmesi sırasında instruction encoding yapılarının detaylı şekilde analiz edilmesi gerekmiştir.

Bu süreç mühendislik problemlerine araştırma odaklı yaklaşım geliştirilmesini sağlamıştır.

13.5 Kazanılan Mühendislik Yetkinlikleri

Bu proje kapsamında aşağıdaki mühendislik yetkinlikleri geliştirilmiştir:

- Sistem programlama becerileri
- Düşük seviyeli yazılım geliştirme deneyimi
- FPGA tabanlı sistem geliştirme
- Assembly dili geliştirme süreçleri
- Modüler yazılım mimarisi tasarımı
- Teknik hata yönetimi
- Test ve doğrulama süreçleri
- Takım çalışması koordinasyonu

Ayrıca proje geliştirme süreci boyunca yeni teknolojilere hızlı şekilde adapte olabilme ve bağımsız teknik araştırma yapabilme becerileri geliştirilmiştir.

Bu kazanımlar mühendislik alanında yaşam boyu öğrenme yaklaşımının önemli bir parçasını oluşturmaktadır.

14. SONUÇ

Bu proje kapsamında PicoRV32 işlemcisi için RV32I instruction set architecture yapısını destekleyen assembler ve linker sistemi başarıyla geliştirilmiştir. Geliştirilen sistem assembly seviyesinde yazılmış kaynak kodları object dosyalarına dönüştürmekte ve farklı object modüllerini birleştirerek FPGA üzerinde çalıştırılabilir executable çıktı üretmektedir.

Assembler sistemi içerisinde parser, lexer, instruction encoder, symbol table yönetimi ve object dosya üretim mekanizmaları geliştirilmiştir. Linker sistemi içerisinde ise ESTAB yapısı, external symbol resolution algoritmaları, relocation mekanizmaları ve executable generation süreçleri gerçekleştirilmiştir.

Projede kullanılan two-pass assembler yaklaşımı sayesinde forward reference problemleri başarılı şekilde çözülmüş ve instruction encoding işlemleri güvenilir şekilde gerçekleştirilmiştir. Ayrıca geliştirilen relocation sistemi branch ve jump instruction'larının linker aşamasında doğru şekilde güncellenmesini sağlamıştır.

Assembler sistemi aşağıdaki temel özellikleri desteklemektedir:

- RV32I instruction encoding
- Symbol table yönetimi
- Two-pass assembler algoritması
- Object dosya üretimi
- Hata yönetim mekanizmaları
- Direktif desteği

Linker sistemi ise aşağıdaki mekanizmaları gerçekleştirmektedir:

- DEF ve REF kayıtlarının analiz edilmesi
- ESTAB oluşturulması
- External symbol resolution işlemleri
- Relocation algoritmaları
- HEX/memory generation süreçleri
- FPGA executable üretimi

Proje kapsamında geliştirilen object dosya yapısı modüler sistem geliştirme yaklaşımını desteklemektedir. MAIN.obj, LOGIC.obj ve IODRV.obj modülleri bağımsız şekilde derlenmiş ve linker sistemi tarafından başarıyla birleştirilmiştir.

FPGA doğrulama süreçleri sırasında PicoRV32 işlemcisi üzerinde executable kod başarıyla çalıştırılmıştır. GTKWave analizleri instruction fetch işlemleri, branch execution süreçleri ve memory erişim mekanizmalarının doğru şekilde çalıştığını göstermiştir.

Ayrıca proje kapsamında gerçekleştirilen hata yönetimi testleri sayesinde assembler sisteminin syntax hataları, duplicate label problemleri, geçersiz direktif kullanımları ve adres taşma hatalarını güvenilir şekilde tespit ettiği doğrulanmıştır.

Bu proje sayesinde aşağıdaki mühendislik kazanımları elde edilmiştir:

- Sistem programlama deneyimi
- Düşük seviyeli yazılım geliştirme becerileri
- RISC-V mimarisi bilgisi
- FPGA tabanlı sistem geliştirme deneyimi
- Relocation algoritmaları geliştirme deneyimi
- Modüler yazılım mimarisi tasarımı
- Teknik test ve doğrulama süreçleri
- Takım çalışması koordinasyonu

Projede geliştirilen sistem eğitim ve akademik amaçlı assembler/linker geliştirme süreçleri açısından önemli mühendislik deneyimi sağlamıştır.

Gelecekte gerçekleştirilebilecek geliştirmeler aşağıdaki gibidir:

- Daha geniş RISC-V instruction desteği
- ELF object format desteği
- Optimize edilmiş linker algoritmaları
- Pipeline tabanlı instruction analizleri
- Gerçek FPGA kartları üzerinde donanımsal doğrulama süreçleri
- Gelişmiş hata ayıklama araçları
- GUI tabanlı gelişmiş assembler ortamı

Sonuç olarak geliştirilen assembler ve linker sistemi RV32I tabanlı assembly programlarının object dosyalarına dönüştürülmesi, relocation işlemlerinin gerçekleştirilmesi ve FPGA üzerinde çalıştırılabilir executable çıktı oluşturulması açısından başarılı şekilde çalışmaktadır.

(PÇ6, PÇ7, PÇ13, PÇ17)

15. PROGRAM ÇIKTILARI (PÇ) DEĞERLENDİRMESİ

PÇ6 – Mühendislik Problemlerini Analiz Etme ve Çözme

Bu proje kapsamında assembler, linker, relocation ve external symbol resolution gibi karmaşık sistem programlama problemleri analiz edilmiş ve mühendislik çözümleri geliştirilmiştir. Özellikle branch relocation işlemleri, symbol management yapıları ve FPGA executable generation süreçleri sistematik şekilde tasarlanmıştır.

PÇ7 – Modern Mühendislik Araçlarını Kullanabilme

Proje geliştirme sürecinde Verilog simülasyon araçları, GTKWave analiz sistemleri ve FPGA tabanlı geliştirme ortamları kullanılmıştır. Ayrıca assembler ve linker sistemleri modern yazılım geliştirme yaklaşımları ile geliştirilmiştir.

PÇ8 – Takım Çalışması Yürütebilme

Proje dört kişilik ekip tarafından geliştirilmiştir. Modüler yazılım geliştirme yaklaşımı kullanılarak görev dağılımı yapılmış ve teknik koordinasyon süreçleri yürütülmüştür.

PÇ9 – Teknik İletişim ve Raporlama

Proje kapsamında teknik raporlama süreçleri gerçekleştirilmiş, sistem mimarisi, algoritmalar ve test süreçleri akademik formatta dokümanite edilmiştir.

PÇ10 – Etik ve Hukuksal Sorumluluk

Projede açık kaynak sistemler kullanılmış ve lisans kurallarına uygun hareket edilmiştir. Ayrıca akademik etik kurallarına dikkat edilmiştir.

PÇ12 - Yaşam Boyu Öğrenme

RISC-V mimarisi, relocation algoritmaları, FPGA sistemleri ve assembler/linker yapıları üzerine araştırmalar gerçekleştirilmiş ve yeni teknolojiler öğrenilmiştir.

PÇ13 - Mühendislik Çözümünün Etkileri

Geliştirilen assembler ve linker sistemi açık kaynak FPGA tabanlı sistem geliştirme süreçlerine katkı sağlayabilecek modüler bir altyapı sunmaktadır.

PÇ17 - Güncel Teknolojileri Takip Edebilme

Projede RISC-V mimarisi, PicoRV32 işlemcisi ve FPGA tabanlı modern sistem geliştirme teknolojileri kullanılmıştır.

16. KAYNAKÇA

- ❖ Patterson, D. A., & Hennessy, J. L. (2021). *Computer organization and design RISC-V edition*. Morgan Kaufmann.
- ❖ Levine, J. R. (2000). *Linkers and loaders*. Morgan Kaufmann Publishers.
- ❖ Stallings, W. (2019). *Computer organization and architecture* (11th ed.). Pearson Education.
- ❖ Mano, M. M., & Ciletti, M. D. (2017). *Digital design* (6th ed.). Pearson.
- ❖ Bhasker, J. (2008). *A Verilog HDL primer* (3rd ed.). Star Galaxy Publishing.
- ❖ IEEE Standards Association. (2020). *IEEE standard for Verilog hardware description language*. IEEE.
- ❖ RISC-V International. (2024). *The RISC-V instruction set manual: Volume I: Unprivileged ISA*. Retrieved from <https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html>
- ❖ RISC-V International. (2024). *RISC-V ratified specifications*. Retrieved from <https://riscv.org/specifications/ratified/>
- ❖ YosysHQ. (2024). *PicoRV32 RISC-V CPU core*. GitHub repository. Retrieved from <https://github.com/YosysHQ/picorv32>
- ❖ YosysHQ. (2024). *PicoRV32 Verilog implementation*. GitHub repository. Retrieved from <https://github.com/YosysHQ/picorv32/blob/main/picorv32.v>
- ❖ YosysHQ. (2024). *PicoSoC README documentation*. GitHub repository. Retrieved from <https://github.com/YosysHQ/picorv32/blob/master/picosoc/README.md>
- ❖ RISC-V ISA Manual Documentation Project. (2024). *RISC-V ISA manual repository*. GitHub repository. Retrieved from <https://github.com/riscv/riscv-isa-manual>
- ❖ Lawrie, D. (2024). *PicoSoC FPGA usage guide*. Retrieved from <https://lawrie.github.io/blackicemxbook/PicoSoC/PicoSoC.html>
- ❖ Grughuhler. (2024). *PicoRV32 FPGA example project*. GitHub repository. Retrieved from https://github.com/grughuhler/picorv32_tang_nano_20k
- ❖ Google Developers. (2024). *RISC-V assembly programmer information*. Retrieved from <https://developers.google.com/coral/guides/hardware/riscv-instr-set>